

# 深度学习入门基于python

---

## 深度学习入门基于python

### 1.感知机

- 1.1 概念
- 1.2 感知机的实现
- 1.3 导入权重和偏置
- 1.4 感知机的局限性
- 1.5 多层感知机

### 2.神经网络

- 2.1 神经网络概念
- 2.2 激活函数
  - 2.2.1 sigmoid函数
  - 2.2.2 ReLU函数
- 2.3 多维数组的运算
  - 2.3.1 矩阵乘法
- 2.4 三层神经网络的实现
  - 2.4.1 符号规定
  - 2.4.2 各层间信号传递的实现
  - 2.4.3 相关代码总结
- 2.5 输出层设计
  - 2.5.1 恒等函数和softmax函数
  - 2.5.2 softmax函数的注意事项
  - 2.5.3 softmax函数特征
  - 2.5.4 输出层的神经元数量
- 2.6 手写数字识别
  - 2.6.1 MNIST数据集
  - 2.6.2 神经网络的推理处理
  - 2.6.3 批处理

### 3.神经网络的学习

- 3.1 从数据中学习
  - 3.1.1 数据驱动
  - 3.1.2 训练数据和测试数据
- 3.2 损失函数
  - 3.2.1 均方误差
  - 3.2.2 交叉熵误差
  - 3.2.3 mini-batch学习
  - 3.2.4 mini-batch版交叉熵误差的实现
  - 3.2.5 为何要设定损失函数
- 3.3 数值微分
  - 3.3.1 导数
  - 3.3.2 数值微分的例子
  - 3.3.3 偏导
- 3.4 梯度
  - 3.4.1 梯度法
  - 3.4.2 神经网络的梯度
- 3.5 学习算法
  - 3.5.1 两层神经网络的类
  - 3.5.2 mini-batch的实现
  - 3.5.3 基于测试数据的评价

## 第4章 误差反向传播法

- 4.1 计算图
  - 4.1.1 用计算图求解
  - 4.1.2 局部计算

- 4.1.3 为何用计算图解题
- 4.2 链式法则
  - 4.2.1 计算图的反向传播
  - 4.2.2 链式法则和计算图
- 4.3 反向传播
  - 4.3.1 加法节点的反向传播
  - 4.3.2 乘法节点的反向传播
- 4.4 简单层的实现
  - 4.4.1 乘法层的实现
  - 4.4.2 加法层的实现
- 4.5 激活函数层的实现
  - 4.5.1 ReLU层
  - 4.5.2 Sigmoid层
- 4.6 Affine/Softmax层的实现
  - 4.6.1 Affine层
  - 4.6.2 批版本的Affine层
  - 4.6.3 Softmax-with-Loss层
- 4.7 误差反向传播法的实现
  - 4.7.1 神经网络学习的全貌图
  - 4.7.2 对应误差反向传播法的神经网络的实现
  - 4.7.3 误差反向传播法的梯度确认
  - 4.7.4 使用误差反向传播法的学习

## 第5章 与学习相关的技巧

- 5.1 参数的更新
  - 5.1.1 SGD
  - 5.1.2 Momentum
  - 5.1.3 AdaGrad
  - 5.1.4 Adam
- 5.2 权重的初始值
  - 5.2.1 可以将权重初始值设为0吗
  - 5.2.2 隐藏层的激活值的分布
  - 5.2.3 ReLU的权重初始值
- 5.3 Batch Normalization
  - 5.3.1 Batch Normalization算法
- 5.4 正则化
  - 5.4.1 过拟合
  - 5.4.2 权值衰减
  - 5.4.3 Dropout
- 5.5 超参数的验证
  - 5.5.1 验证数据
  - 5.5.2 超参数的最优化

## 第6章 卷积神经网络(CNN)

- 6.1 整体结构
- 6.2 卷积层
  - 6.2.1 全连接层存在的问题
  - 6.2.2 卷积运算
  - 6.2.3 填充
  - 6.2.4 步幅
  - 6.2.5 三维数据的卷积运算
  - 6.2.6 结合方块思考
  - 6.2.7 批处理
- 6.3 池化层
- 6.4 卷积层和池化层的实现
  - 6.4.1 基于im2col的展开
  - 6.4.2 卷积层的实现
  - 6.4.3 池化层的实现
- 6.5 CNN的实现
- 6.6 具有代表性的CNN

6.6.1 LeNet

6.6.2 AlexNet

## 第7章 深度学习

7.1 加深网络

7.2 深度学习的小历史

7.2.1 ImageNet

7.2.2 VGG

7.2.3 GoogleNet

7.2.4 ResNet

7.3 深度学习的未来

# 1.感知机

## 1.1 概念

感知机接收多个输入信号，输出一个信号。

感知机的信号只有“流/不流”（1/0）两种取值。

- 0：不传递信号
- 1：传递信号

$$y = \begin{cases} 0 & (w_1x_1 + w_2x_2 \leq \theta) \\ 1 & (w_1x_1 + w_2x_2 > \theta) \end{cases}$$

- w：权重，控制各个信号
- x：输入信号
- y：输出信号
- $\theta$ ：阈值，神经元被激活临界值

机器学习的课题是将**决定参数值**的工作交由计算机自动进行。

**学习**是确定合适的参数的过程，而人要做的是思考感知机的构造（模型），并把训练数据交给计算机。

## 1.2 感知机的实现

```
1 def AND(x1, x2):
2     # 设置权重和阈值
3     w1, w2, theta = 0.5, 0.5, 0.7
4     tmp = x1*w1 + x2*w2
5     if tmp <= theta:
6         return 0
7     elif tmp > theta:
8         return 1
```

## 1.3 导入权重和偏置

$$y = \begin{cases} 0 & (b + w_1x_1 + w_2x_2 \leq 0) \\ 1 & (b + w_1x_1 + w_2x_2 > 0) \end{cases}$$

- b：偏置，用来调整神经元被激活的容易程度的参数
  - 若b为-0.1，则只要输入信号的加权总和超过0.1，神经元就会被激活

- w: 权重, 控制各个信号
- x: 输入信号
- y: 输出信号
- $\theta$ : 阈值, 神经元被激活临界值

示例代码:

```

1  '''含有权重和偏置的感知机'''
2  # 与门
3  import numpy as np
4  def AND(x1, x2):
5      # 设置输入信号
6      x = np.array([x1, x2])
7      # 设置权重
8      w = np.array([0.5, 0.5])
9      # 设置偏置
10     b = -0.7
11     # sum函数用来计算权重和输入乘积后的和
12     tmp = np.sum(w*x) + b
13     if tmp <= 0:
14         return 0
15     else:
16         return 1

```

或门和非门实现只在于权重参数的值

```

1  '''非门实现'''
2
3  import numpy as np
4  def NAND(x1, x2):
5      # 设置输入信号
6      x = np.array([x1, x2])
7      # 设置权重
8      w = np.array([-0.5, -0.5]) #仅权重和偏置和与门不同
9      # 设置偏置
10     b = 0.7
11     # sum函数用来计算权重和输入乘积后的和
12     tmp = np.sum(w*x) + b
13     if tmp <= 0:
14         return 0
15     else:
16         return 1
17
18
19  '''或门'''
20  import numpy as np
21  def OR(x1, x2):
22      # 设置输入信号
23      x = np.array([x1, x2])
24      # 设置权重
25      w = np.array([0.5, 0.5]) #仅权重和偏置和与门不同
26      # 设置偏置
27      b = -0.2
28      # sum函数用来计算权重和输入乘积后的和
29      tmp = np.sum(w*x) + b

```

```

30     if tmp <= 0:
31         return 0
32     else:
33         return 1

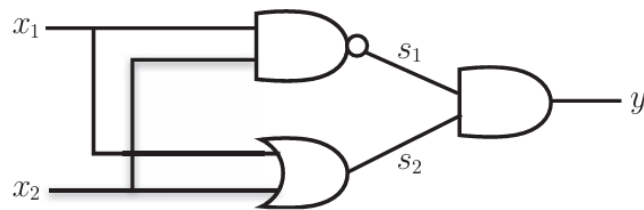
```

## 1.4 感知机的局限性

感知机的局限性就在于它只能表示由一条**直线分割**的空间。

## 1.5 多层感知机

异或门是一种多层结构的神经网络。这里，将最左边的一列称为第0层，中间的一列称为第1层，最右边的一列称为第2层。



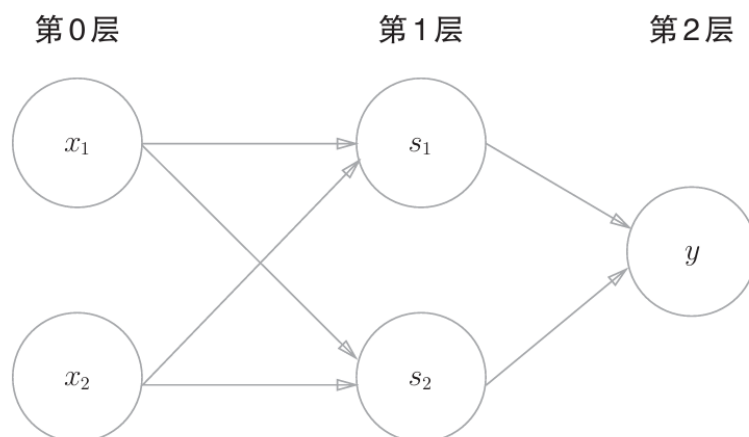
代码实现：

```

1  '''异或门实现'''
2  def XOR(x1, x2):
3      s1 = NAND(x1, x2) # 非门
4      s2 = OR(x1, x2) # 或门
5      y = AND(s1, s2) # 与门
6      return y

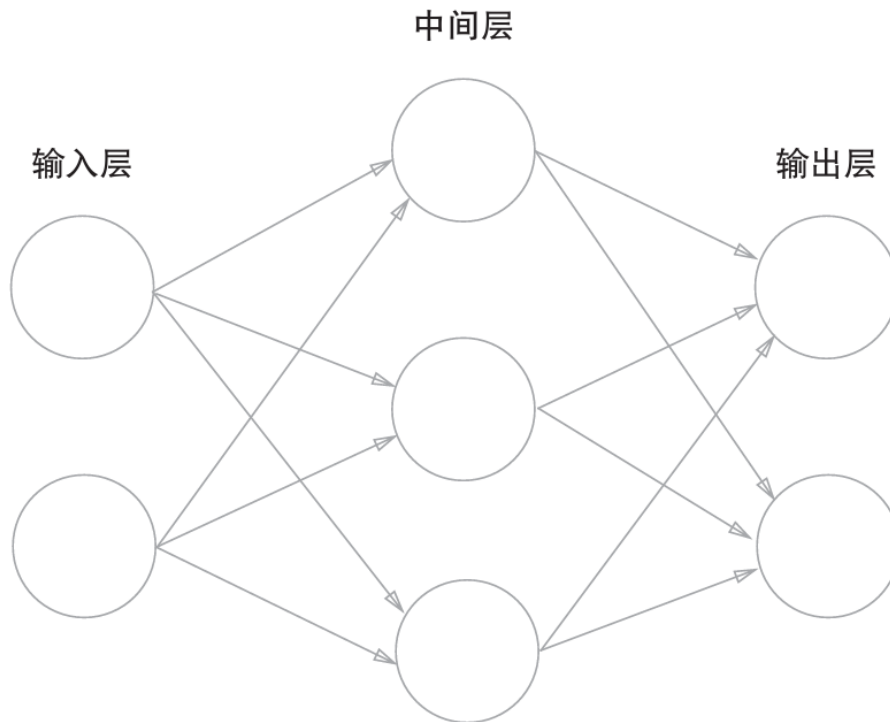
```

使用感知机表示异或门



## 2.神经网络

### 2.1 神经网络概念



两层网络

- 输入层：第0层
- 中间层：第1层，又叫隐藏层
- 输出层：第2层

激活函数 $h()$

$$a = b + w_1x_1 + w_2x_2$$

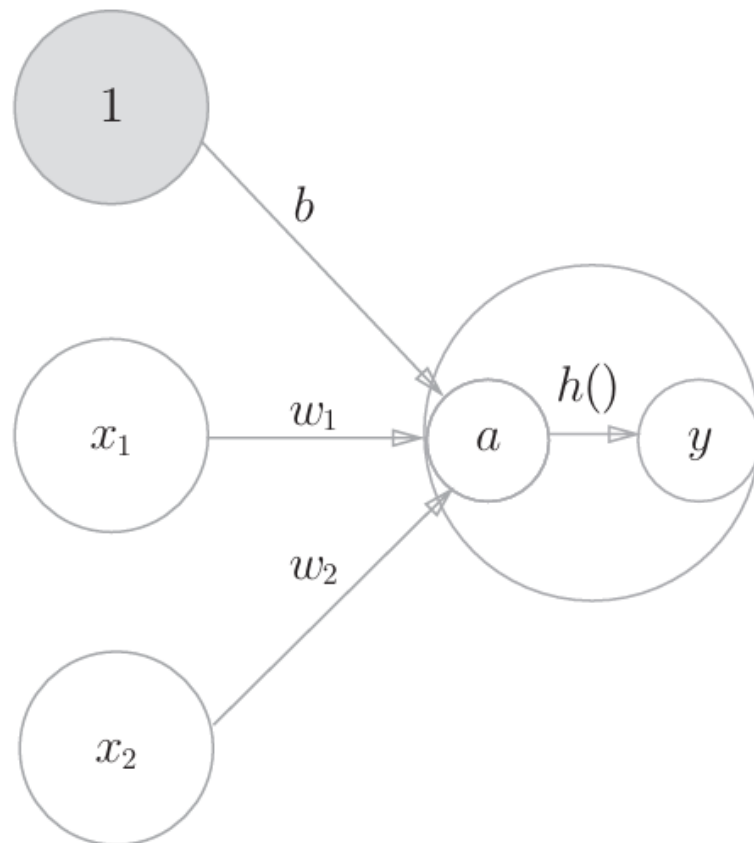
$$y = h(a)$$

- $a$ ：加权输入信号和偏置的总和
- $h()$ ：激活函数

实现图如下：

其中  $a$ 和 $y$ 可称为“节点”（和神经元含义相同）

○：神经元



## 2.2 激活函数

激活函数以阈值为界，一旦输入超过阈值，就切换输出。这样的函数称为“阶跃函数”。神经网络的激活函数必须使用**非线性函数**。

### 2.2.1 sigmoid函数

**sigmoid函数表达式**

$$h(x) = \frac{1}{1+e^{-x}}$$

$$h(x) = \frac{1}{1 + \exp(-x)}$$

- $\exp(-x)$ :  $e^{-x}$

#### 阶跃函数的图形

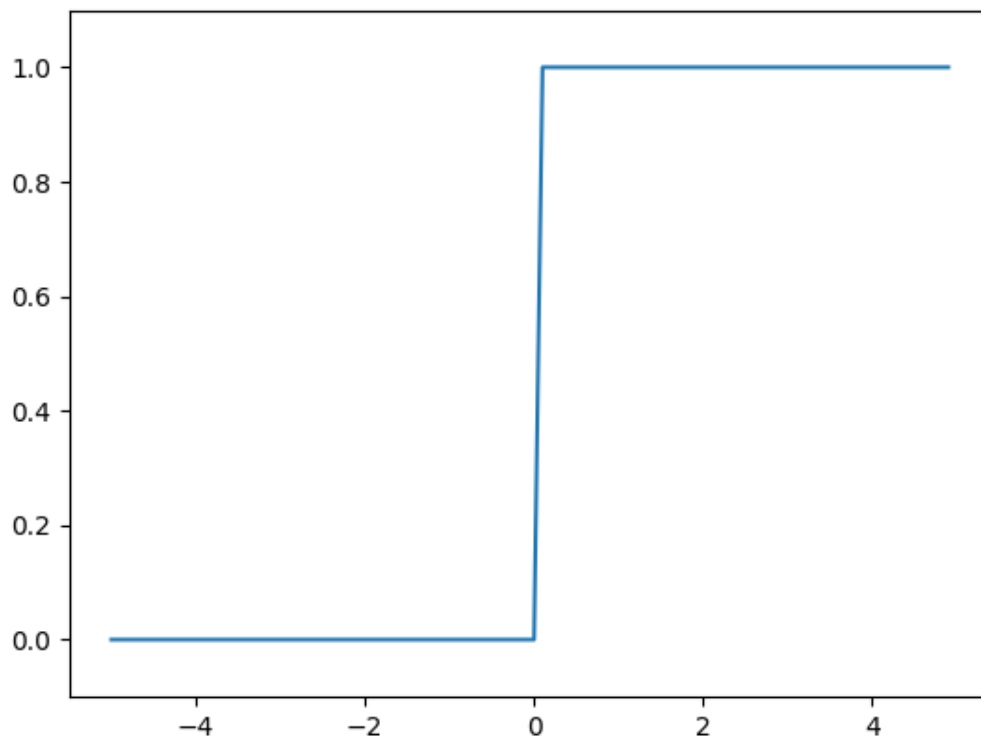
阶跃函数以0为界，输出从0切换为1（或者从1切换为0）。它的值呈阶梯式变化，所以称为阶跃函数



```

1  '''阶跃函数的图形'''
2  import numpy as np
3  import matplotlib.pyplot as plt
4
5  def step_function(x):
6      return np.array(x>0, dtype=np.int_)
7  # 在-5.0到5.0的范围内，以0.1为单位
8  x = np.arange(-5.0, 5.0, 0.1)
9  y = step_function(x)
10 plt.plot(x, y)
11 plt.ylim(-0.1, 1.1) #指定y轴的范围
12 plt.show()

```



### sigmoid函数的实现

```

1  '''sigmoid函数实现'''
2  import numpy as np
3  def sigmoid(x):
4      # 就是那个数学公式
5      return 1/(1+np.exp(-x))
6  x = np.array([-1.0, 1.0, 2.0])
7  print(sigmoid(x))

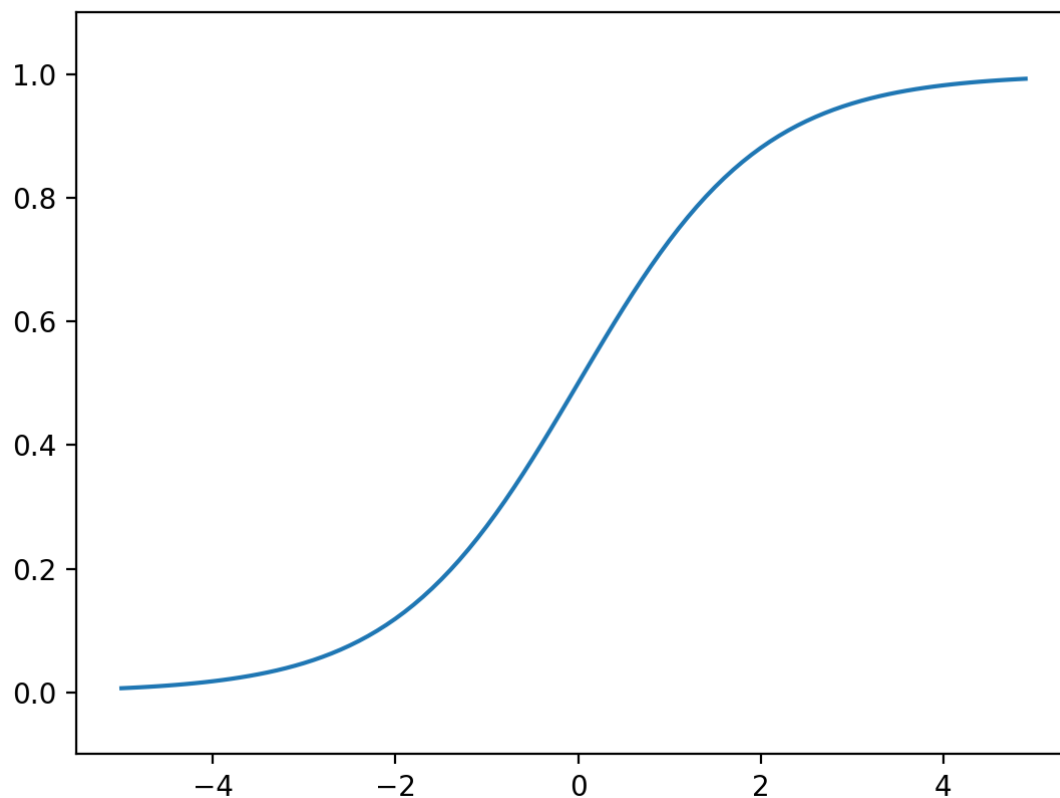
```

### sigmoid函数图像

```

1  '''sigmoid图像'''
2  import numpy as np
3  import matplotlib.pyplot as plt
4
5  def sigmoid(x):
6      return 1/(1+np.exp(-x))
7
8  x = np.arange(-5.0, 5.0, 0.1)
9  y = sigmoid(x) # 使用sigmoid函数
10 plt.plot(x, y)
11 plt.ylim(-0.1, 1.1) #指定y轴的范围
12 plt.show()

```



## 2.2.2 ReLU函数

ReLU函数

- 输入大于0时，直接输出该值
- 输入小于等于0时，输出0

$$h(x) = \begin{cases} x & (x > 0) \\ 0 & (x \leq 0) \end{cases}$$

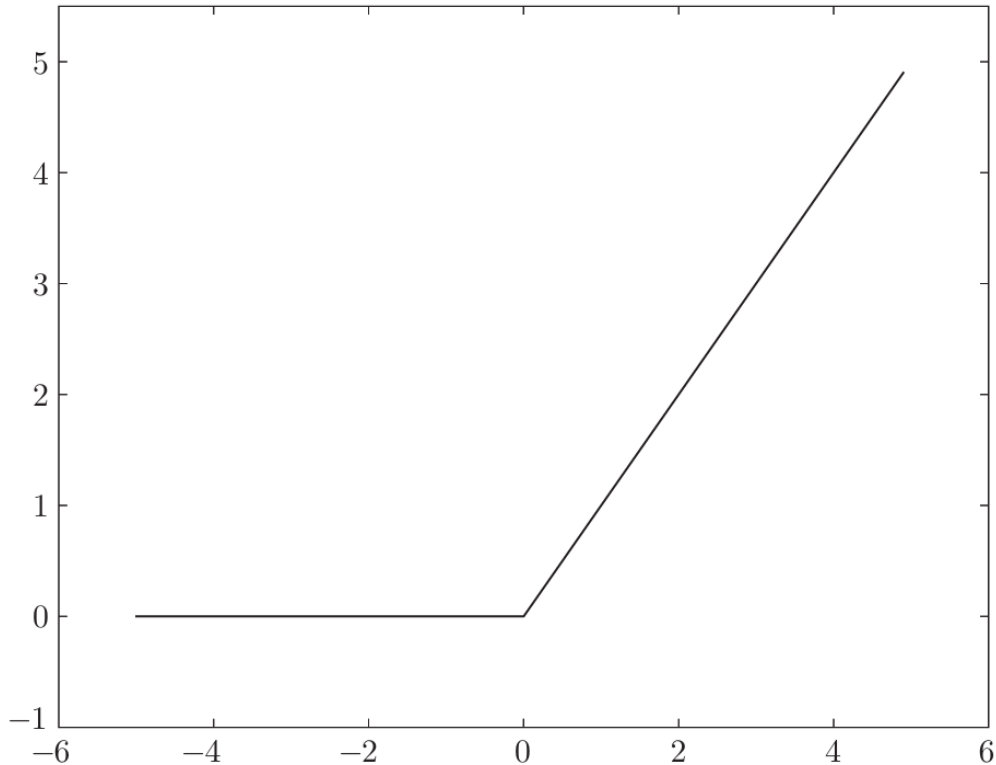
代码实现

```

1  '''ReLU函数实现'''
2  import numpy as np
3  def relu(x):
4      # 大于0, 则返回x; 否则返回0
5      return np.maximum(0, x)

```

函数图像如下:



## 2.3 多维数组的运算

数组的维度: 通过 `np.ndim(变量)` 获得

数组的形状: 通过 `变量.shape` 来获得, 返回一个元组

```

1  import numpy as np
2  A = np.array([[1, 2], [3, 4]])
3  n = np.ndim(A) # 获取A的维度
4  s = A.shape # 获取A的形状
5  print(n) # 输出维度
6  print(s) # 输出形状

```

### 2.3.1 矩阵乘法

矩阵乘法: 使用 `np.dot(矩阵1, 矩阵2)` 计算

```

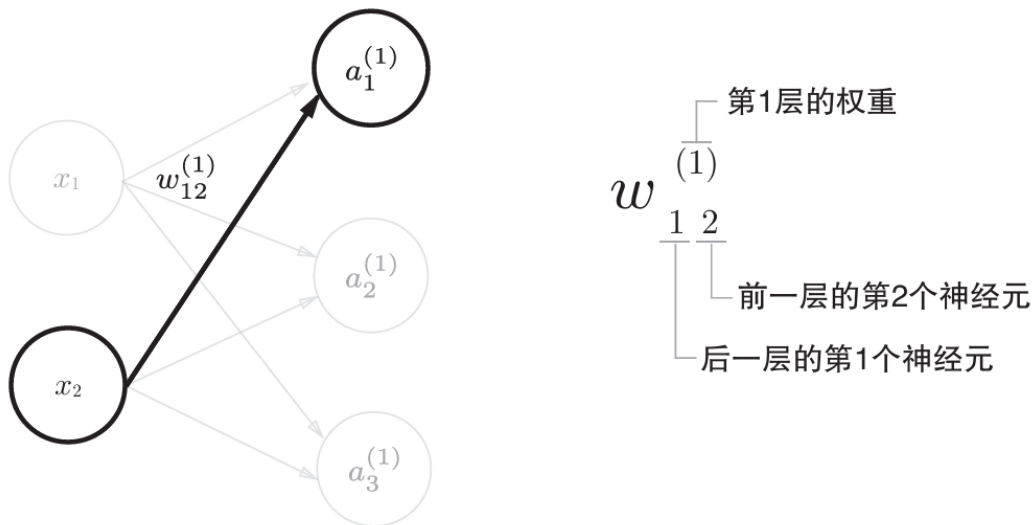
1  '''矩阵乘法'''
2  import numpy as np
3  A = np.array([[1, 2], [3, 4]])
4  B = np.array([[5, 6], [7, 8]])
5  C = np.dot(A, B) # 计算矩阵A和B的乘积
6  print(C)

```

## 2.4 三层神经网络的实现

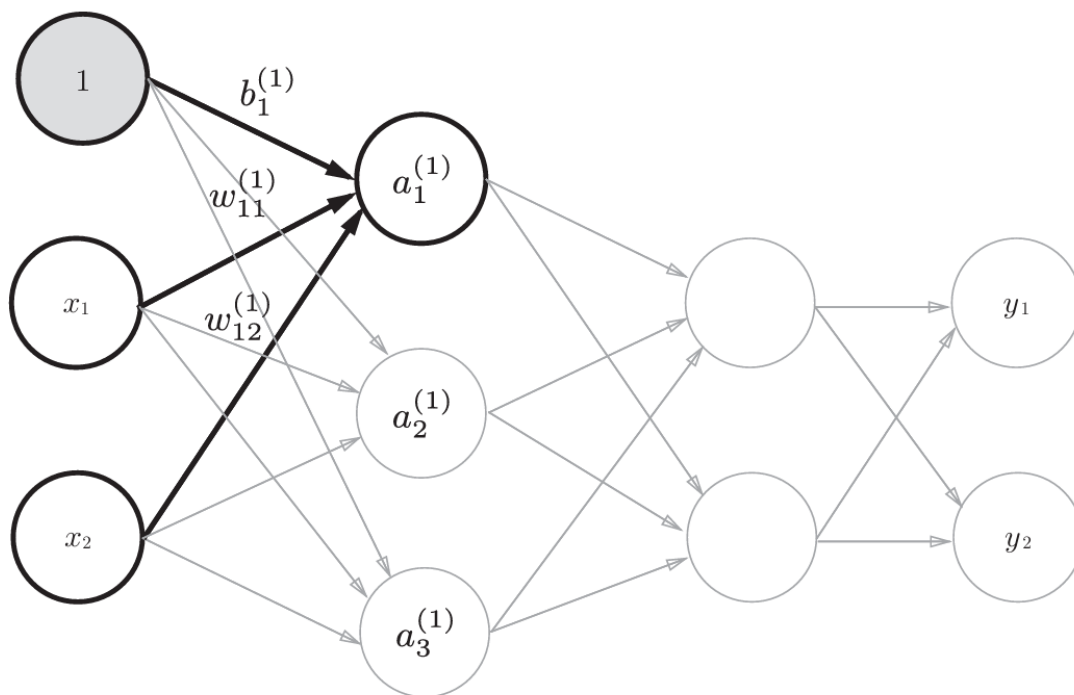
### 2.4.1 符号规定

如下图规定：



### 2.4.2 各层间信号传递的实现

从输入层到第1层的第1个神经元的传递过程，标黑的过程



$$a_1^{(1)} = w_{11}^{(1)} x_1 + w_{12}^{(1)} x_2 + b_1^{(1)}$$

各个公式表示：

$$\mathbf{A}^{(1)} = \mathbf{X}\mathbf{W}^{(1)} + \mathbf{B}^{(1)} \quad (3.9)$$

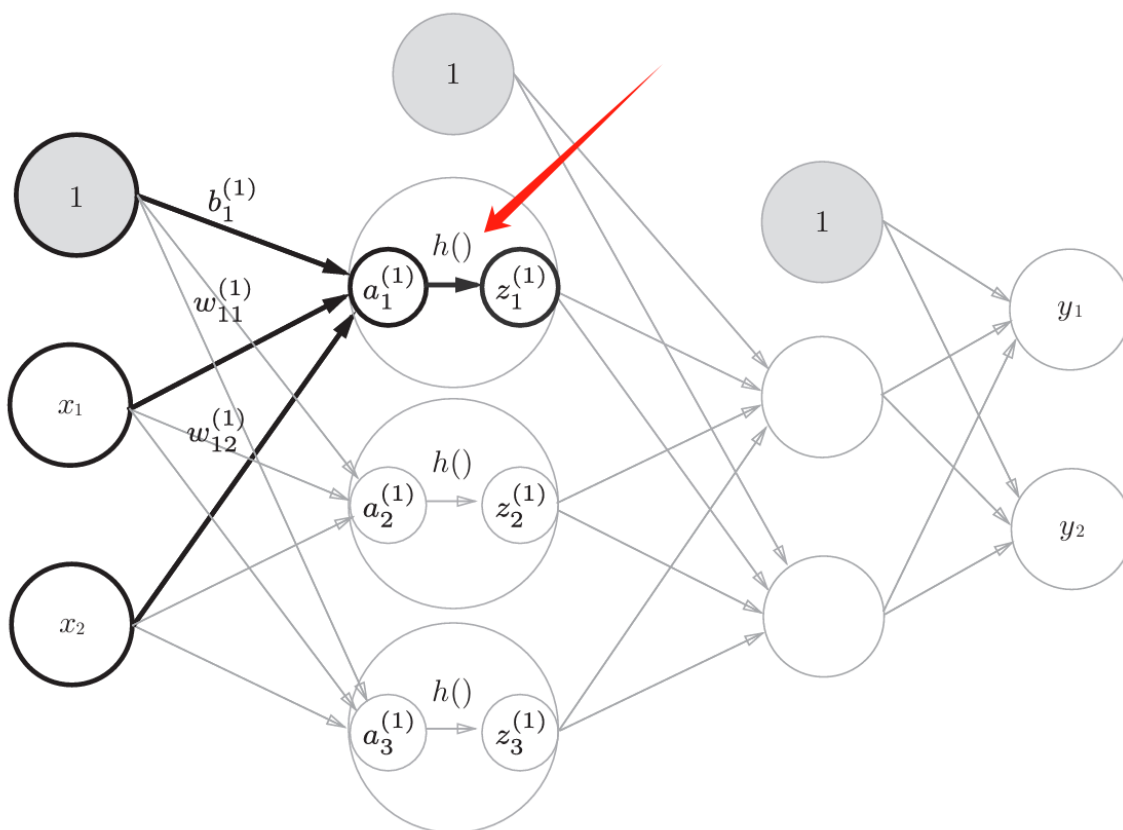
其中， $\mathbf{A}^{(1)}$ 、 $\mathbf{X}$ 、 $\mathbf{B}^{(1)}$ 、 $\mathbf{W}^{(1)}$ 如下所示。

$$\mathbf{A}^{(1)} = \begin{pmatrix} a_1^{(1)} & a_2^{(1)} & a_3^{(1)} \end{pmatrix}, \quad \mathbf{X} = \begin{pmatrix} x_1 & x_2 \end{pmatrix}, \quad \mathbf{B}^{(1)} = \begin{pmatrix} b_1^{(1)} & b_2^{(1)} & b_3^{(1)} \end{pmatrix}$$

$$\mathbf{W}^{(1)} = \begin{pmatrix} w_{11}^{(1)} & w_{21}^{(1)} & w_{31}^{(1)} \\ w_{12}^{(1)} & w_{22}^{(1)} & w_{32}^{(1)} \end{pmatrix}$$

使用sigmoid函数实现第1层的输出

- 隐藏层的加权和（加权信号和偏置的总和）用  $a$  表示
- 被激活函数转换后的信号用  $z$  表示。



示例代码：

```

1  '''第1层的输出'''
2  import numpy as np
3
4  def sigmoid(x):
5      return 1/(1+np.exp(-x))
6
7  # x: 输入层
8  # w1: 权重（从输入层到第1层）
9  # B1: 偏置（从输入层到第1层）
10 x = np.array([1.0, 0.5])
11 w1 = np.array([[0.1, 0.3, 0.5], [0.2, 0.4, 0.6]])
12 B1 = np.array([0.1, 0.2, 0.3])

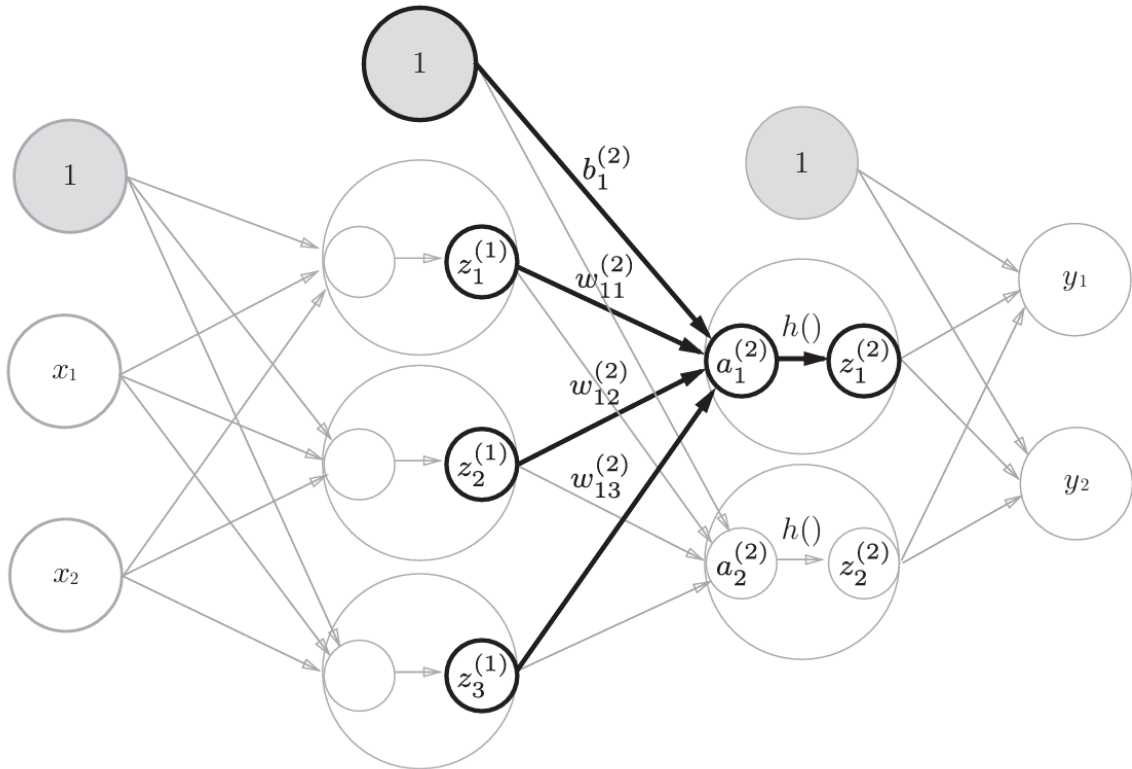
```

```

13 # 分别求出第1层的加权和
14 A1 = np.dot(X, w1) + B1
15
16 # 调用sigmoid函数,计算第1层的输出
17 z1 = sigmoid(A1)
18
19 print(A1)
20 print(z1)

```

从第1层到第2层时, 使用的输入为 Z1 (sigmoid函数计算后的结果)



```

1 '''第1层的输出'''
2 import numpy as np
3
4 def sigmoid(x):
5     return 1/(1+np.exp(-x))
6
7 # x: 输入层
8 # w1: 权重 (从输入层到第1层)
9 # B1: 偏置 (从输入层到第1层)
10 x = np.array([1.0, 0.5])
11 w1 = np.array([[0.1, 0.3, 0.5], [0.2, 0.4, 0.6]])
12 B1 = np.array([0.1, 0.2, 0.3])
13
14 # 分别求出第1层的加权和
15 A1 = np.dot(X, w1) + B1
16
17 # 调用sigmoid函数,计算第1层的输出
18 z1 = sigmoid(A1)
19
20 print(A1)
21 print(z1)
22
23 '''第2层的计算'''
24 # z1: 第1层的输入

```

```

24 # w2: 第1层到第2层的权重
25 # B2: 第1层到第2层的偏置
26 w2 = np.array([[0.1, 0.4], [0.2, 0.5], [0.3, 0.6]])
27 B2 = np.array([0.1, 0.2])
28
29 # 计算第2层的加权和
30 A2 = np.dot(Z1, w2) + B2
31 # 计算第2层的输出
32 z2 = sigmoid(A2)
33 print(A2)
34 print(z2)

```

## 2.4.3 相关代码总结

init\_network()函数“会进行权重和偏置的初始化，并将它们保存在字典变量network中。

- 这个字典变量network中保存了每一层所需的参数（权重和偏置）。

forward()函数：中则封装了将输入信号转换为输出信号的处理过程。

```

1  '''三层神经网络代码'''
2  import numpy as np
3
4  # 定义sigmoid函数
5  def sigmoid(x):
6      return 1/(1+np.exp(-x))
7
8  # 定义恒等函数
9  def identity_function(x):
10     return x
11
12 # 初始化权重和偏置
13 def init_network():
14     network = {}
15     network['w1'] = np.array([[0.1, 0.3, 0.5], [0.2, 0.4, 0.6]])
16     network['b1'] = np.array([0.1, 0.2, 0.3])
17     network['w2'] = np.array([[0.1, 0.4], [0.2, 0.5], [0.3, 0.6]])
18     network['b2'] = np.array([0.1, 0.2])
19     network['w3'] = np.array([[0.1, 0.3], [0.2, 0.4]])
20     network['b3'] = np.array([0.1, 0.2])
21     return network
22
23 # 定义前向函数
24 def forward(network, x):
25     w1, w2, w3 = network['w1'], network['w2'], network['w3']
26     b1, b2, b3 = network['b1'], network['b2'], network['b3']
27
28     # 获取第1层的加权和和输出
29     a1 = np.dot(x, w1) + b1
30     z1 = sigmoid(a1)
31     # 获取第2层的加权和和输出
32     a2 = np.dot(z1, w2) + b2 # 此时 z1: 第1层到第2层的输入
33     z2 = sigmoid(a2)
34     # 获取第3层的加权和和输出
35     a3 = np.dot(z2, w3) + b3
36     y = identity_function(a3) # 调用恒等函数
37

```

```

38     return y
39
40     network = init_network()
41     x = np.array([1.0, 0.5])
42     y = forward(network, x)
43     print(y)

```

## 2.5 输出层设计

神经网络可以用在分类问题和回归问题上，不过需要根据情况改变输出层的激活函数。

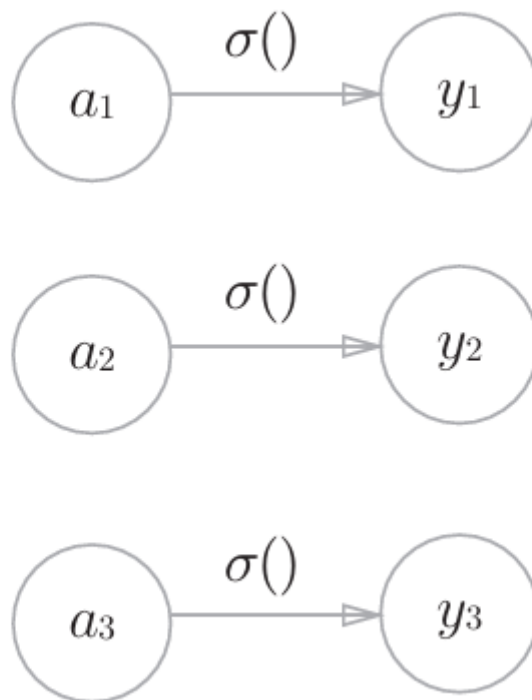
- 回归问题用**恒等函数**
- 分类问题用**softmax函数**

### 2.5.1 恒等函数和softmax函数

#### (1) 恒等函数

恒等函数会将输入按原样输出，对于输入的信息，不加以任何改动地直接输出。因此，在输出层使用恒等函数时，输入信号会原封不动地被输出。

神经网络图表示如下：



#### (2) softmax函数

softmax函数如下：

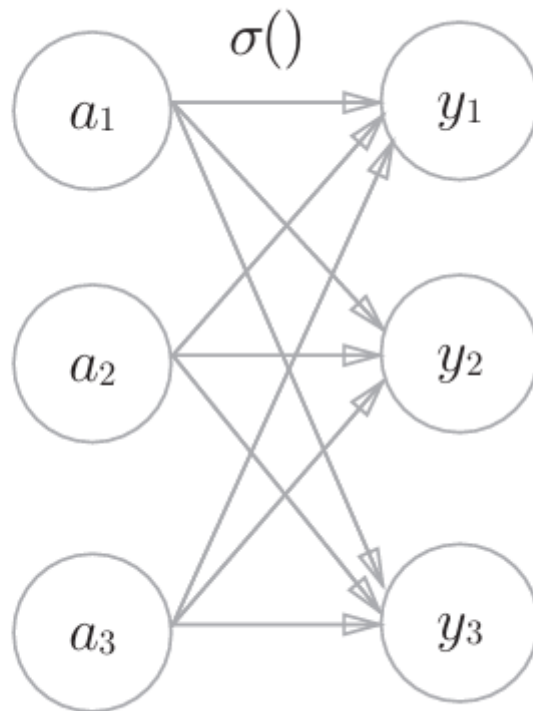
$$y_k = \frac{\exp(a_k)}{\sum_{i=1}^n \exp(a_i)}$$



假设输出层共有 $n$ 个神经元，计算第 $k$ 个神经元的输出 $y_k$

- 分子：输入信号  $a_k$  的指数函数
- 分母：所有输入信号的指数函数的和

神经网络图如下：



代码实现：

```
1  '''softmax函数实现'''
2  # 一般实现
3  import numpy as np
4  a = np.array([0.3, 2.9, 4.0])
5  exp_a = np.exp(a) # 指数函数
6
7  sum_exp_a = np.sum(exp_a) # 指数函数的和
8
9  # 计算softmax函数
10 y = exp_a / sum_exp_a
11 print(y)
```

函数形式实现

```
1  '''函数形式实现'''
2  # 注：这个函数可能会出现溢出，改进后的代码参考下一个softmax函数的注意事项中的代码
3  import numpy as np
4  def softmax(a):
5      exp_a = np.exp(a)
6      sum_exp_a = np.sum(exp_a)
7      y = exp_a / sum_exp_a
8
9      return y
```

## 2.5.2 softmax函数的注意事项

缺陷：溢出问题，指数函数运算，值会变得很大

改进：

softmax 函数的实现可以像式 (3.11) 这样进行改进。

$$\begin{aligned} y_k &= \frac{\exp(a_k)}{\sum_{i=1}^n \exp(a_i)} = \frac{C \exp(a_k)}{C \sum_{i=1}^n \exp(a_i)} \\ &= \frac{\exp(a_k + \log C)}{\sum_{i=1}^n \exp(a_i + \log C)} \\ &= \frac{\exp(a_k + C')}{\sum_{i=1}^n \exp(a_i + C')} \end{aligned} \quad (3.11)$$

代码实现：

```
1  '''改进的softmax函数'''
2  import numpy as np
3  def softmax(a):
4      c = np.max(a) # 取最大值
5      exp_a = np.exp(a - c) # 溢出对策
6      sum_exp_a = np.sum(exp_a)
7      y = exp_a / sum_exp_a
8      return y
```

## 2.5.3 softmax函数特征

softmax函数的输出是0.0到1.0之间的实数。

- 并且，softmax 函数的输出值的总和是1。
- 正因为输出值总和为1，我们才可以把softmax函数的输出解释为“概率”。

一般而言，神经网络只把**输出值最大**的神经元所对应的类别作为识别结果。

- 即便使用softmax函数，输出值最大的神经元的位置也不会变。
- 因此，神经网络在进行分类时，输出层的softmax函数可以省略。

示例代码：

```

1 import numpy as np
2 def softmax(a):
3     c = np.max(a) # 取最大值
4     exp_a = np.exp(a - c) # 溢出对策
5     sum_exp_a = np.sum(exp_a)
6     y = exp_a / sum_exp_a
7     return y
8
9 a = np.array([0.3, 2.9, 4.0])
10 y = softmax(a)
11 print(y) # 输出每个神经元的概率
12 s = np.sum(y)
13 print(y) # 和为1

```

## 2.5.4 输出层的神经元数量

输出层的神经元数量需要根据待解决的问题来决定。

- 对于分类问题，输出层的神经元数量一般设定为类别的数量。

## 2.6 手写数字识别

### 2.6.1 MNIST数据集

MNIST数据集是由0到9的数字图像构成的（图3-24）。训练图像有6万张，测试图像有1万张，这些图像可以用于学习和推理

MNIST的图像数据是28像素×28像素的灰度图像（1通道），各个像素的取值在0到255之间。每个图像数据都相应地标有“7”“2”“1”等标签。

读入MNIST数据：

```

1 '''手写数字识别'''
2 import sys, os
3 sys.path.append(os.pardir) # 为了导入父目录中的文件而进行的设定
4 from dataset.mnist import load_mnist
5
6 # 第1次调用会花费几分钟
7 # load_mnist函数以“(训练图像,训练标签), (测试图像, 测试标签)”的形式返回读入的MNIST数据
8 # 第1个参数normalize设置是否将输入图像正规化为0.0~1.0的值, 如果将该参数设置为False, 则
  输入图像的像素会保持原来的0~255。
9 # 第2个参数flatten设置是否展开输入图像（变成一维数组）。如果将该参数设置为False, 则输入
  图像为1x28x28的三维数组; 若设置为True, 则输入图像会保存为由784个元素构成的一维数组。
10 # 第3个参数one_hot_label设置是否将标签保存为onehot表示（one-hot representation）。
   one-hot表示是仅正确解标签为1, 其余皆为0的数组, 就像[0,0,1,0,0,0,0,0,0,0]这样。当
   one_hot_label为False时,
11 # 只是像7、2这样简单保存正确解标签; 当one_hot_label为True时, 标签则保存为one-hot表示。
12 (x_train, t_train), (x_test, t_test) = load_mnist(flatten=True,
   normalize=False)
13
14 # 输出各个数据的形状
15 print(x_train.shape) # x训练集: (60000, 784)
16 print(t_train.shape) # t训练集: (60000,)
17 print(x_test.shape) # x测试集: (10000, 784)
18 print(t_test.shape) # t测试集: (10000,)

```

```

1  '''显示图像'''
2  import sys, os
3  sys.path.append(os.pardir) # 为了导入父目录中的文件而进行的设定
4  import numpy as np
5  from dataset.mnist import load_mnist
6  from PIL import Image
7
8  def img_show(img):
9      # 将Numpy数组的图像数据转换为PIL用的数据
10     pil_img = Image.fromarray(np.uint8(img))
11     pil_img.show()
12
13     # 第1次调用会花费几分钟
14     # flatten=True时读入的图像是以一列（一维）Numpy数组的形式保存的。
15     (x_train, t_train), (x_test, t_test) = load_mnist(flatten=True,
16     normalize=False)
17
18     img = x_train[0]
19     label = t_train[0]
20
21     print(img.shape) # (784,)
22     img = img.reshape(28, 28) # 把图像的形状变成原来的尺寸
23     print(img.shape)
24
25     img_show(img) # 显示图像

```

## 2.6.2 神经网络的推理处理

神经网络的输入层有784个神经元，输出层有10个神经元。

- 输入层的784这个数字来源于图像大小的 $28 \times 28 = 784$
- 输出层的10这个数字来源于10类别分类（数字0到9，共10类别）。

### 推理过程

- 首先获得MNIST数据集，生成网络。
- 接着，用for语句逐一取出保存在x中的图像数据，用predict()函数进行分类。
- 然后，我们取出这个概率列表中的最大值的索引（第几个元素的概率最高），作为预测结果。
- 最后，比较神经网络所预测的答案和正确解标签，将回答正确的概率作为识别精度。

```

1  '''神经网络推理实现'''
2  import numpy as np
3  from dataset.mnist import load_mnist
4  import pickle
5
6  # 定义sigmoid函数
7  def sigmoid(x):
8      return 1/(1+np.exp(-x))
9
10     # 定义softmax函数
11     def softmax(a):
12         c = np.max(a) # 取最大值
13         exp_a = np.exp(a - c) # 溢出对策
14         sum_exp_a = np.sum(exp_a)
15         y = exp_a / sum_exp_a

```

```

16     return y
17
18 # 获取数据
19 def get_data():
20     (x_train, t_train), (x_test, t_test) = load_mnist(normalize=True,
21     flatten=True, one_hot_label=False)
22     return x_test, t_test
23
24 # 初始化数据
25 def init_network():
26     with open("sample_weight.pkl", 'rb') as f:
27         network = pickle.load(f)
28     return network
29
30 # 预测数据
31 def predict(network, x):
32     w1, w2, w3 = network['w1'], network['w2'], network['w3']
33     b1, b2, b3 = network['b1'], network['b2'], network['b3']
34
35     a1 = np.dot(x, w1) + b1
36     z1 = sigmoid(a1)
37
38     a2 = np.dot(z1, w2) + b2
39     z2 = sigmoid(a2)
40
41     a3 = np.dot(z2, w3) + b3
42     y = softmax(a3)
43
44     return y
45
46 x, t = get_data()
47 network = init_network()
48
49 accuracy_cnt = 0
50 for i in range(len(x)):
51     y = predict(network, x[i])
52     p = np.argmax(y) # 获取概率最高的元素的索引
53     if p == t[i]:
54         accuracy_cnt += 1
55 print("Accuracy:" + str(float(accuracy_cnt)/len(x)))

```

- 把数据限定到某个范围内的处理称为**正规化** (normalization) 。
- 对神经网络的输入数据 进行某种既定的转换称为**预处理** (pre-processing)

### 2.6.3 批处理

现在我们来考虑打包输入多张图像的情形。比如，我们想用predict() 函数一次性打包处理100张图像

```

1  '''神经网络批处理实现'''
2  import numpy as np
3  from dataset.mnist import load_mnist
4  import pickle
5
6  # 定义sigmoid函数
7  def sigmoid(x):
8      return 1/(1+np.exp(-x))

```

```

9
10 # 定义softmax函数
11 def softmax(a):
12     c = np.max(a) # 取最大值
13     exp_a = np.exp(a - c) # 溢出对策
14     sum_exp_a = np.sum(exp_a)
15     y = exp_a / sum_exp_a
16     return y
17
18 # 获取数据
19 def get_data():
20     (x_train, t_train), (x_test, t_test) = load_mnist(normalize=True,
21     flatten=True, one_hot_label=False)
22     return x_test, t_test
23
24 # 初始化数据
25 def init_network():
26     with open("sample_weight.pkl", 'rb') as f:
27         network = pickle.load(f)
28     return network
29
30 # 预测数据
31 def predict(network, x):
32     w1, w2, w3 = network['w1'], network['w2'], network['w3']
33     b1, b2, b3 = network['b1'], network['b2'], network['b3']
34
35     a1 = np.dot(x, w1) + b1
36     z1 = sigmoid(a1)
37
38     a2 = np.dot(z1, w2) + b2
39     z2 = sigmoid(a2)
40
41     a3 = np.dot(z2, w3) + b3
42     y = softmax(a3)
43
44     return y
45
46
47 # 批处理
48 x, t = get_data()
49 network = init_network()
50
51 batch_size = 100 # 批数量
52 accuracy_cnt = 0
53
54 for i in range(0, len(x), batch_size):
55     x_batch = x[i:i+batch_size]
56     y_batch = predict(network, x_batch)
57     p = np.argmax(y_batch, axis=1)
58     accuracy_cnt += np.sum(p == t[i:i+batch_size])
59 print("Accuracy:" + str(float(accuracy_cnt)/len(x)))

```

## 3.神经网络的学习

---

这里所说的“学习”是指从训练数据中 自动获取最优权重参数的过程。

### 3.1 从数据中学习

---

所谓“从数据中学习”，是指可以由数据自动决定权重参数的值。

#### 3.1.1 数据驱动

利用有效数据解决问题

- 一种方案是，先从图像中提取特征量，再用机器学习技术学习这些特征量的模式。
- 这里所说的“特征量”是指可以 从输入数据（输入图像）中准确地提取本质数据（重要的数据）的转换器。

神经网络的优点是对所有的问题都可以用同样的流程来解决。

#### 3.1.2 训练数据和测试数据

机器学习一般过程：

- 首先，使用**训练数据**进行学习，寻找最优的参数
- 然后，使用**测试数据**评价训练得到的模型的实际能力。

将数据分为训练数据和测试数据的原因：

- 为了正确评价模型的**泛化能力**，就必须划分训练数据和测试数据。
- 训练数据也称**监督数据**

**泛化能力**是指处理未被观察过的数据（不包含在训练数据中的数据）的能力。获得泛化能力是机器学习的最终目标。

## 3.2 损失函数

---

神经网络的学习通过某个指标表示现在的状态。然后，以这个指标为基准，寻找最优权重参数。

神经网络的学习中所用的指标称为**损失函数**（loss function）。

- 这个损失函数可以使用任意函数
- 但一般用均方误差和交叉熵误差等

**损失函数**是表示神经网络性能的“恶劣程度”的指标

- 即当前的神经网络对监督数据在多大程度上不拟合，在多大程度上不一致。

#### 3.2.1 均方误差

均方误差是损失函数之一，公式如下：

$$E = \frac{1}{2} \sum_k (y_k - t_k)^2$$

- $y_k$ : 表示神经网络的输出
- $t_k$ : 表示监督数据
- $k$ : 表示数据的维度

通过运行下面代码我们知道

第1个例子中的损失函数值更小，和监督数据之间的误差较小

```

1  '''均方误差'''
2  import numpy as np
3
4  # 定义均方误差函数
5  # 参数y和t是numpy参数
6  def mean_squared_error(y, t):
7      return 0.5 * np.sum((y-t)**2)
8
9  # 第1个例子
10 # t中标签正确为1，下面中正确标签为数字2
11 t = [0, 0, 1, 0, 0, 0, 0, 0, 0, 0]
12 # 数字2的概率最高情况(0.6)
13 y = [0.1, 0.05, 0.6, 0.0, 0.05, 0.1, 0.0, 0.1, 0.0, 0.0]
14
15 s = mean_squared_error(np.array(y), np.array(t))
16 print(s) #0.09750000000000003
17
18 # 第2个例子
19 # 数字7的概率最高
20 y = [0.1, 0.05, 0.1, 0.0, 0.05, 0.1, 0.0, 0.6, 0.0, 0.0]
21 s = mean_squared_error(np.array(y), np.array(t))
22 print(s) #0.5975

```

### 3.2.2 交叉熵误差

交叉熵误差 (cross entropy error) 也经常被用作损失函数。交叉熵误差如下式所示。

$$E = - \sum_k t_k \log y_k$$

- $y_k$ : 神经网络的输出
- $t_k$ : 正确解标签



- $t_k$ 中只有正确解标签的索引为1
- 其他均为0

**交叉熵误差的值**是由正确解标签所对应的输出结果决定的

- 第一个例子中，正确解标签对应的输出为0.6，此时的交叉熵误差大约为0.51。
- 第二个例子中，正确解标签对应的输出为0.1的低值，此时的交叉熵误差大约为2.3。

结果是我们可以预料到的，正好是对应 $t$ 中值为1的索引，投射到 $y$ 中的概率中计算所得到的值

```

1  '''交叉熵误差'''
2  import numpy as np
3
4  # 定义交叉熵误差函数
5  # 参数y和t是NumPy数组
6  def cross_entropy_error(y, t):
7      # 值delta:添加一个微小值可以防止负无限大的发生。
8      delta = 1e-7
9      return -np.sum(t*np.log(y+delta))
10
11 # 第1个例子
12 # t中正确解标签为2
13 t = [0, 0, 1, 0, 0, 0, 0, 0, 0, 0]
14 # 神经网络的输出
15 y = [0.1, 0.05, 0.6, 0.0, 0.05, 0.1, 0.0, 0.1, 0.0, 0.0]
16
17 c = cross_entropy_error(np.array(y), np.array(t))
18 print(c) #0.510825457099338
19
20 # 第2个例子
21 y = [0.1, 0.05, 0.1, 0.0, 0.05, 0.1, 0.0, 0.6, 0.0, 0.0]
22
23 c = cross_entropy_error(np.array(y), np.array(t))
24 print(c) #2.302584092994546

```

### 3.2.3 mini-batch学习

使用训练数据进行学习，严格来说，就是针对训练数据计算损失函数的值，找出使该值尽可能小的参数。

- 因此，计算损失函数时必须将所有的训练数据作为对象。
- 也就是说，如果训练数据有100个的话，我们就要把这100个损失函数的总和作为学习的指标。

使用交叉熵误差计算所有训练数据的损失函数的总和公式如下：

$$E = -\frac{1}{N} \sum_n \sum_k t_{nk} \log y_{nk}$$

- $N$ ： $N$ 个数据
- $t_{nk}$ ：表示第 $n$ 个数据的第 $k$ 个元素的值
- $y_{nk}$ ：是神经网络的输出

- $t_{nk}$ : 是监督数据

`np.random.choice()`: 随机抽取

我们只需指定这些随机选出的索引，取出mini-batch，然后使用 这个mini-batch计算损失函数即可。

### 3.2.4 mini-batch版交叉熵误差的实现

同时处理单个数据和批量数据（数据作为batch集中输入）两种情况的函数。

```
1  '''mini-batch版交叉熵误差的实现'''
2  import numpy as np
3
4  # 定义函数
5  # y是神经网络的输出
6  # t是监督数据
7  def cross_entropy_error(y, t):
8      if y.ndim == 1:
9          t = t.reshape(1, t.size)
10         y = y.reshape(1, y.size)
11         batch_size = y.shape[0]
12         return -np.sum(t * np.log(y+1e-7)) / batch_size
```

当监督数据是标签形式

```
1  '''mini-batch版交叉熵误差的实现'''
2  import numpy as np
3
4  # 定义函数
5  # y是神经网络的输出
6  # t是监督数据
7
8  # 当监督数据是标签形式（非one-hot表示，而是像“2”“ 7”这样的标签）时
9
10 def cross_entropy_error(y, t):
11     if y.ndim == 1:
12         t = t.reshape(1, t.size)
13         y = y.reshape(1, y.size)
14         batch_size = y.shape[0]
15         return -np.sum(t * np.log(y[np.arange(batch_size), t] + 1e-7)) /
16             batch_size
```

### 3.2.5 为何要设定损失函数

为了找到使损失函数的值尽可能小的地方

- 需要计算参数的导数（确切地讲是梯度）
- 然后以这个导数为指引，逐步更新参数的值。

对权重参数的损失函数求导，表示的是“如果稍微改变这个权重参数的值，损失函数的值会如何变化”。

- 如果导数的值为负，通过使该权重参数向正方向改变，可以减小损失函数的值
- 如果导数的值为正，则通过使该权重参数向负方向改变，可以减小损失函数的值。
- 当导数的值为0时，无论权重参数向哪个方向变化，损失函数的值都不会改变，此时该权重参数的更新会停在此处

在进行神经网络的学习时，不能将识别精度作为指标。因为如果以识别精度为指标，则参数的导数在绝大多数地方都会变为0。

## 3.3 数值微分

梯度法使用梯度的信息决定前进的方向。

### 3.3.1 导数

利用微小的差分求导数的过程称为数值微分

使用下面微分求导公式：

$$\frac{df(x)}{dx} = \lim_{h \rightarrow 0} \frac{f(x+h) - f(x)}{h}$$

```
1  '''导数'''
2
3  def numerical_diff(f, x):
4      h = 1e-4 # 0.0001
5      return (f(x+h) - f(x-h)) / (2*h)
```

### 3.3.2 数值微分的例子

$f(x) = 0.01x^2 + 0.1x$ 在 $x=5$ 时的导数如下所示

```
1  '''导数'''
2
3  def numerical_diff(f, x):
4      h = 1e-4 # 0.0001
5      return (f(x+h) - f(x-h)) / (2*h)
6
7  '''数值微分示例'''
8
9  def function_1(x):
10     return 0.01*x**2 + 0.1*x
11
12  # 计算 y = 0.01x^2 + 0.1x的导数
13  # x=5时的导数
14  d = numerical_diff(function_1, 5)
15  print('x=5时的导数: ', d)
```

### 3.3.3 偏导

$$f(x_0, x_1) = x_0^2 + x_1^2$$

求偏导：

```
1  '''导数'''
2
3  def numerical_diff(f, x):
```

```

4     h = 1e-4 # 0.0001
5     return (f(x+h) - f(x-h)) / (2*h)
6
7
8 # 求偏导
9 # 求x0 = 3, x1 = 4时, 关于x0的偏导
10 def function_tmp1(x0):
11     return x0*x0 + 4.0**2.0
12
13 d = numerical_diff(function_tmp1, 3.0)
14 print('x0的偏导为: ',d)

```

## 3.4 梯度

**梯度**: 由全部变量的偏导数汇总而成的向量

`np.zeros_like(x)` 会生成一个形状和x相同、所有元素都为0的数组。

更严格地讲, 梯度指示的方向是各点处的**函数值减小最多**的方向

```

1 '''梯度'''
2 def numerical_gradient(f, x):
3     # f: 要求导的函数, 输入为向量x, 输出为标量
4     # x: numpy数组, 表示自变量向量
5     # return: 与x同形状的梯度向量
6     h = 1e-4 # 0.0001
7     grad = np.zeros_like(x) # 生成和x形状相同的数组
8
9     # 遍历 x 的每一个元素, 逐个计算偏导
10    for idx in range(x.size):
11        # 保存原始值, 便于后续还原
12        tmp_val = x[idx]
13
14        # f(x+h)的计算
15        x[idx] = tmp_val + h # 仅对第 idx 个元素加上 h
16        fxh1 = f(x)
17
18        # f(x-h)的计算
19        x[idx] = tmp_val - h
20        fxh2 = f(x)
21
22        # 中心差分近似偏导数
23        grad[idx] = (fxh1 - fxh2) / (2*h)
24        # 还原 x[idx], 避免影响下一轮计算
25        x[idx] = tmp_val # 还原值
26    return grad
27
28 # 定义函数
29 def function_2(x):
30     return x[0]*x[0] + x[1]*x[1]
31
32 ti = numerical_gradient(function_2, np.array([3.0, 4.0]))
33 print('函数在(3, 4)处的梯度: ', ti)

```

### 3.4.1 梯度法

神经网络必须在学习时找到最优参数（权重和偏置），也就是指损失函数取最小值时的参数。

函数的极小值、最小值以及被称为**鞍点**（saddle point）的地方，梯度为0。

虽然梯度的方向并不一定指向最小值，但沿着它的方向能够最大限度地减小函数的值。因此，在寻找函数的最小值（或者尽可能小的值）的位置的任务中，要以梯度的信息为线索，决定前进的方向。

通过不断地沿梯度方向前进，逐渐减小函数值的过程就是**梯度法**。

梯度法是解决机器学习中最优化问题的常用方法，特别是在神经网络的学习中经常被使用。

- 寻找最小值的梯度法称为**梯度下降法**（gradient descent method）
- 寻找最大值的梯度法称为**梯度上升法**（gradient ascent method）

$$x_0 = x_0 - \eta \frac{\partial f}{\partial x_0}$$
$$x_1 = x_1 - \eta \frac{\partial f}{\partial x_1}$$

$\eta$ 表示**更新量**，在神经网络的学习中，称为**学习率**（learning rate）。

- 学习率决定在一次学习中，应该学习多少，以及在多大程度上更新参数。

代码中的相关参数：

- 参数f是要进行最优化的函数
- init\_x是初始值
- lr是学习率learning rate
- step\_num是梯度法的重复次数。

```
1  '''梯度下降法'''
2  def gradient_descent(f, init_x, lr=0.01, step_num=100):
3      x = init_x
4
5      for i in range(step_num):
6          grad = numerical_gradient(f, x)
7          x -= lr * grad
8
9      return x
10
11 def function_3(x):
12     return x[0]**2 + x[1]**2
13
14 init_x = np.array([-3.0, 4.0])
15
16 ti1 = gradient_descent(function_3, init_x=init_x, lr=0.1, step_num=100)
17 print('最小值: ',ti1)
```

实验结果表明

- 学习率过大的话，会发散成一个很大的值
- 学习率过小的话，基本上没怎么更新就结束了。
- 也就是说，设定合适的学习率 是一个很重要的问题。

学习率这样的参数称为**超参数**

### 3.4.2 神经网络的梯度

神经网络的学习也要求梯度。这里所说的梯度是指损失函数关于**权重参数的梯度**。

$$\mathbf{W} = \begin{pmatrix} w_{11} & w_{12} & w_{13} \\ w_{21} & w_{22} & w_{23} \end{pmatrix}$$
$$\frac{\partial L}{\partial \mathbf{W}} = \begin{pmatrix} \frac{\partial L}{\partial w_{11}} & \frac{\partial L}{\partial w_{12}} & \frac{\partial L}{\partial w_{13}} \\ \frac{\partial L}{\partial w_{21}} & \frac{\partial L}{\partial w_{22}} & \frac{\partial L}{\partial w_{23}} \end{pmatrix}$$

```
1  '''神经网络的梯度'''
2  import sys, os
3  sys.path.append(os.pardir)
4  import numpy as np
5  from common.functions import softmax, cross_entropy_error
6  from common.gradient import numerical_gradient
7
8  # 定义simpleNet类
9  class simpleNet:
10     def __init__(self):
11         self.w = np.random.randn(2, 3) # 用高斯分布实现初始化
12
13     def predict(self, x):
14         return np.dot(x, self.w)
15     def loss(self, x, t):
16         z = self.predict(x)
17         y = softmax(z)
18         loss = cross_entropy_error(y, t)
19
20         return loss
21
22 net = simpleNet()
23 print(net.w) # 权重参数
24
25
26 x = np.array([0.6, 0.9])
27 p = net.predict(x)
28 print(p)
29
```

```
30 print(np.argmax(p)) # 最大值的索引
31
32 t = np.array([0, 0, 1]) # 正确解标签
33 print(net.loss(x, t))
```

## 3.5 学习算法

神经网络的学习步骤如下所示

一共4个步骤：

### 步骤1 (mini-batch)

- 从训练数据中随机选出一部分数据，这部分数据称为mini-batch。我们的目标是减小mini-batch的损失函数的值。

### 步骤2 (计算梯度)

- 为了减小mini-batch的损失函数的值，要求出各个权重参数的梯度。梯度表示损失函数的值减小最多的方向。

### 步骤3 (更新参数)

- 将权重参数沿梯度方向进行微小更新。

### 步骤4 (重复)

- 重复步骤1、步骤2、步骤3。

这个方法通过梯度下降法更新参数，不过因为这里使用的数据是随机选择的mini batch数据，所以又称为 **随机梯度下降法** (stochastic gradient descent)

### 3.5.1 两层神经网络的类

#### (1) TwolayerNet类中使用的变量

params：保存神经网络的参数的字典变量（实例变量）

- params['W1']是第1层的权重，params['b1']是第1层的偏置。
- params['W2']是第2层的权重，params['b2']是第2层的偏置

grads：保存梯度的字典变量（numerical\_gradient()方法的返回值）

- grads['W1']是第1层权重的梯度，grads['b1']是第1层偏置的梯度。
- grads['W2']是第2层权重的梯度，grads['b2']是第2层偏置的梯度

#### (2) TwoLayerNet类的方法

\_\_init\_\_(self, input\_size, hidden\_size, output\_size)：进行初始化

- 参数从头开始依次表示输入层的神经元数、隐藏层的神经元数、输出层的神经元数

predict(self, x)：进行识别（推理）。

- 参数x是图像数据

loss(self, x, t)：计算损失函数的值。

- 参数 x是图像数据
- t是正确解标签（后面3个方法的参数也一样）

| <code>accuracy(self, x, t)</code>           | 计算识别精度                                                    |
|---------------------------------------------|-----------------------------------------------------------|
| <code>numerical_gradient(self, x, t)</code> | 计算权重参数的梯度                                                 |
| <code>gradient(self, x, t)</code>           | 计算权重参数的梯度。 <code>numerical_gradient()</code> 的高速版，将在下一章实现 |

```
1  '''2层神经网络的类'''
2  from two_layer_net import *
3
4  net = TwoLayerNet(input_size=784, hidden_size=100, output_size=10)
5  print(net.params['w1'].shape) # (784, 100)
6  print(net.params['b1'].shape) # (100,)
7  print(net.params['w2'].shape) # (100, 10)
8  print(net.params['b2'].shape) # (10,)
```

### 3.5.2 mini-batch的实现

所谓 mini-batch学习，就是从训练数据中随机选择一部分数据（称为mini-batch），再以这些mini-batch为对象，使用梯度法更新参数的过程。

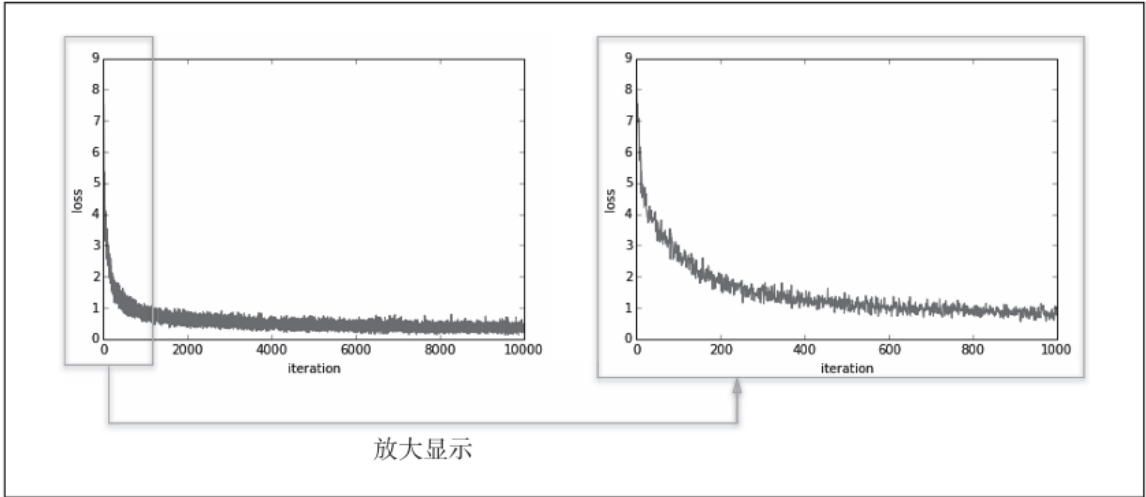


图 4-11 损失函数的推移：左图是 10000 次循环的推移，右图是 1000 次循环的推移

观察上图可以发现随着学习的进行，损失函数的值在不断减小。这是学习正常进行的信号，表示神经网络的权重参数在逐渐拟合数据。

### 3.5.3 基于测试数据的评价

**过拟合**是指，虽然训练数据中的数字图像能被正确辨别，但是不在训练数据中的数字图像却无法被识别的现象。

**epoch**是一个单位。一个epoch表示学习中所有训练数据均被使用过一次时的更新次数。



# 第4章 误差反向传播法

要正确理解误差反向传播法，我个人认为有两种方法：

- 一种是基于数学式
- 另一种是基于计算图

## 4.1 计算图

### 4.1.1 用计算图求解

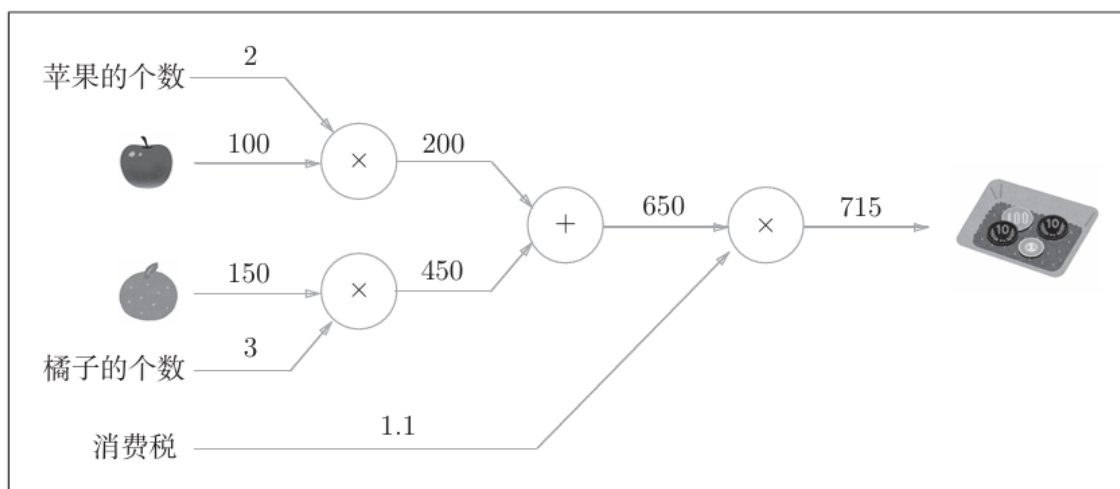
计算图通过节点和箭头表示计算过程。

- 节点用 $\circ$ 表示
- $\circ$ 中是计算的内容。
- 将计算的中间结果写在箭头的上方，表示各个节点的计算结果从左向右传递。

通过问题示例来画图：

太郎在超市买了2个苹果、3个橘子。其中，苹果每个100日元，橘子每个150日元。消费税是10%，请计算支付金额。

问题的计算图如下所示：



**正向传播：**从左向右进行计算

**反向传播：**从右到左，在导数计算中发挥重要作用

### 4.1.2 局部计算

计算图可以集中精力于局部计算。无论全局的计算有多么复杂，各个步骤所要做的就是对象节点的局部计算。虽然局部计算非常简单，但是通过传递它的计算结果，可以获得全局的复杂计算的结果。

### 4.1.3 为何用计算图解题

计算图的两个优点：

- 局部计算。
- 利用计算图可以将中间的计算结果全部保存起来。

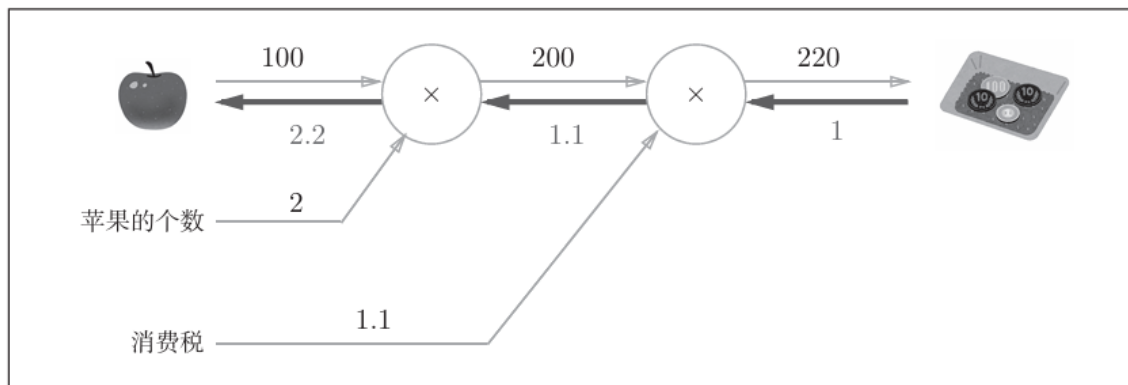
实际上，使用计算图最大的原因是，可以通过反向传播高效计算导数。

这里，假设我们想知道苹果价格的上涨会在多大程度上影响最终的支付金额，即求“支付金额关于苹果的价格的导数”。

设苹果的价格为 $x$ ，支付金额为 $L$ ，则相当于求 $\frac{\partial L}{\partial x}$ 。这个导数的值表示当苹果的价格稍微上涨时，支付金额会增加多少。

计算图如下：

反向传播使用与正方向相反的箭头（粗线）表示。



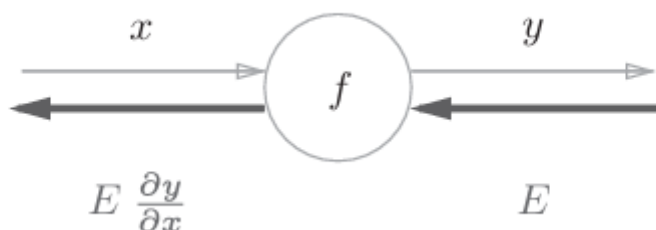
在这个例子中，反向传播从右向左传递导数的值（ $1 \rightarrow 1.1 \rightarrow 2.2$ ）。

从这个结果中可知，“支付金额关于苹果的价格的导数”的值是2.2。这意味着，如果苹果的价格上涨1日元，最终的支付金额会增加2.2日元（严格地讲，如果苹果的价格增加某个微小值，则最终的支付金额将增加那个微小值的2.2倍）。

## 4.2 链式法则

### 4.2.1 计算图的反向传播

假设存在  $y = f(x)$  的计算，这个计算的反向传播如下图：



如图所示，反向传播的计算顺序是，将信号 $E$ 乘以节点的局部导数（ $\frac{\partial y}{\partial x}$ ），然后将结果传递给下一个节点。

- 这里所说的局部导数是指正向传播中 $y=f(x)$ 的导数，也就是 $y$ 关于 $x$ 的导数（ $\frac{\partial y}{\partial x}$ ）
- 把这个局部导数乘以上游传过来的值（本例中为 $E$ ），然后传递给前面的节点。

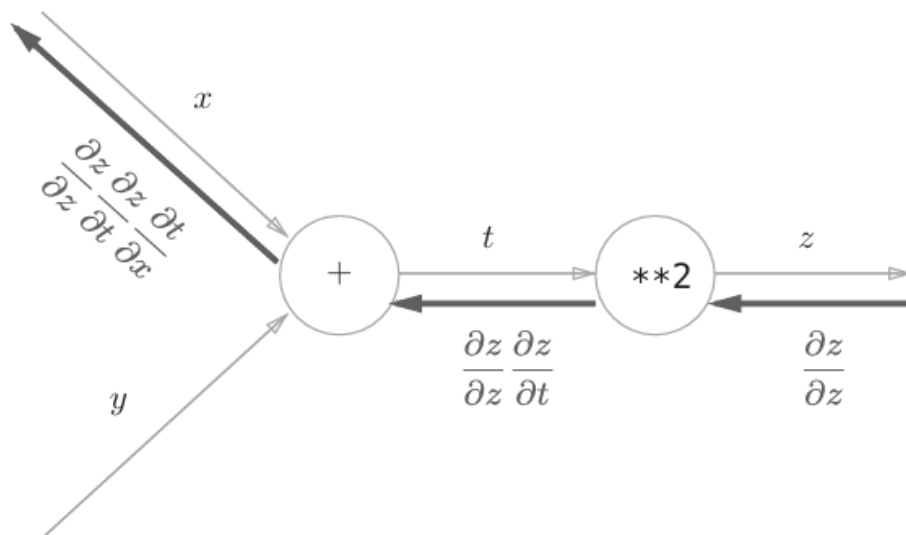
### 4.2.2 链式法则和计算图

公式如下：

$$z = t^2$$

$$t = x + y$$

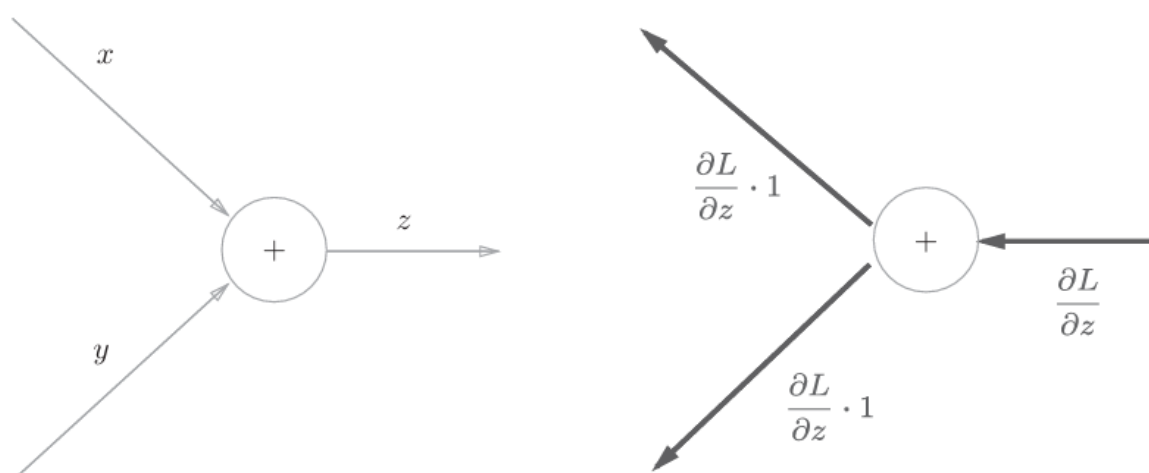
他的链式法则的计算图如下：



## 4.3 反向传播

### 4.3.1 加法节点的反向传播

以  $z = x + y$  为对象观察反向传播



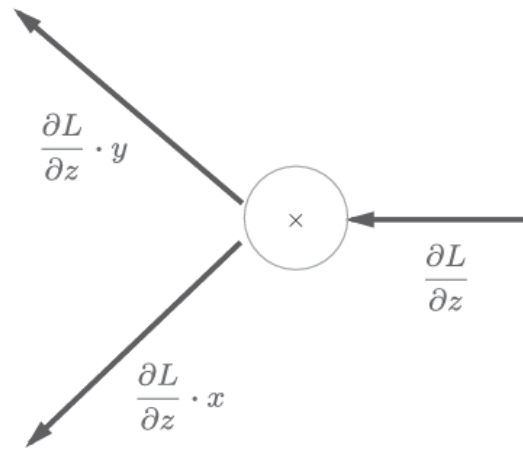
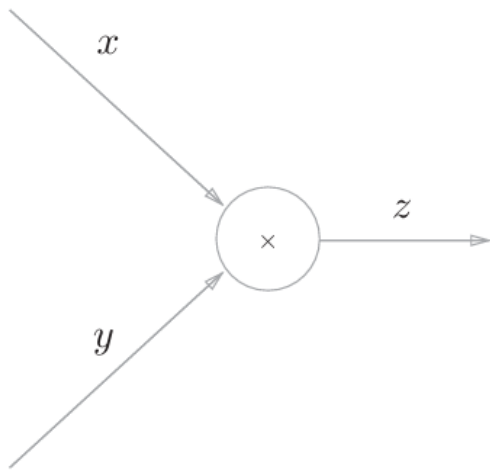
加法节点的反向传播:

- 左图是正向传播
- 右图是反向传播。

### 4.3.2 乘法节点的反向传播

考虑  $z = xy$

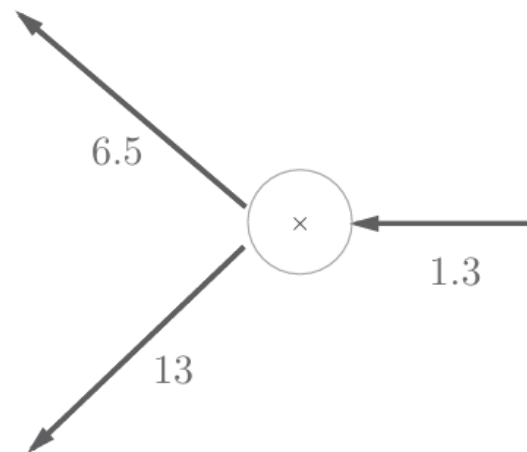
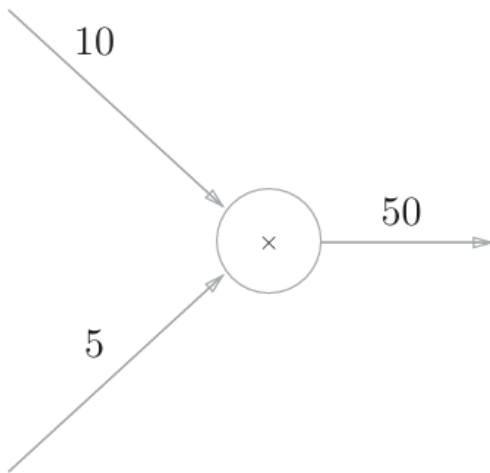
计算图如下:



乘法的反向传播:

- 左图是正向传播
- 右图是反向传播

具体例子:



## 4.4 简单层的实现

### 4.4.1 乘法层的实现

乘法层作为MulLayer类

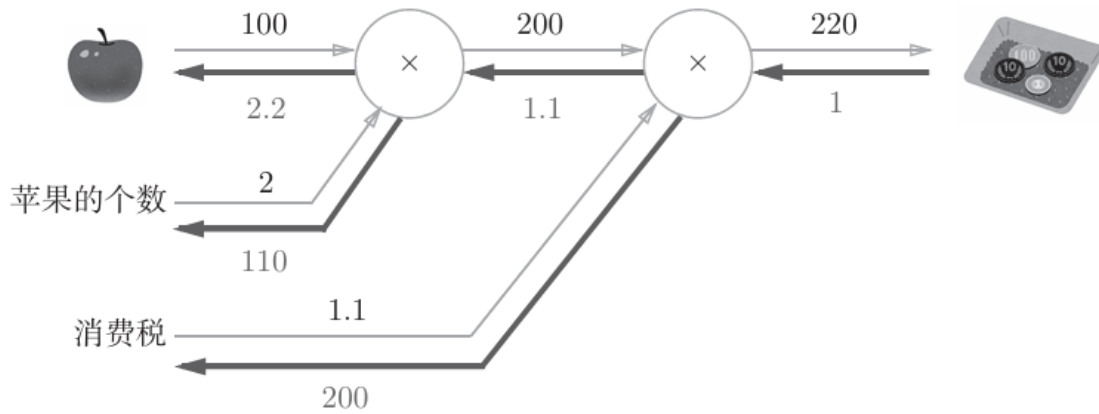
```

1 class MulLayer:
2     def __init__(self):
3         self.x = None
4         self.y = None
5     def forward(self, x, y):
6         self.x = x
7         self.y = y
8         out = x * y
9         return out
10    def backward(self, dout):
11        dx = dout * self.y # 翻转x和y
12        dy = dout * self.x
13        return dx, dy

```

相关示例代码:

计算图如下：



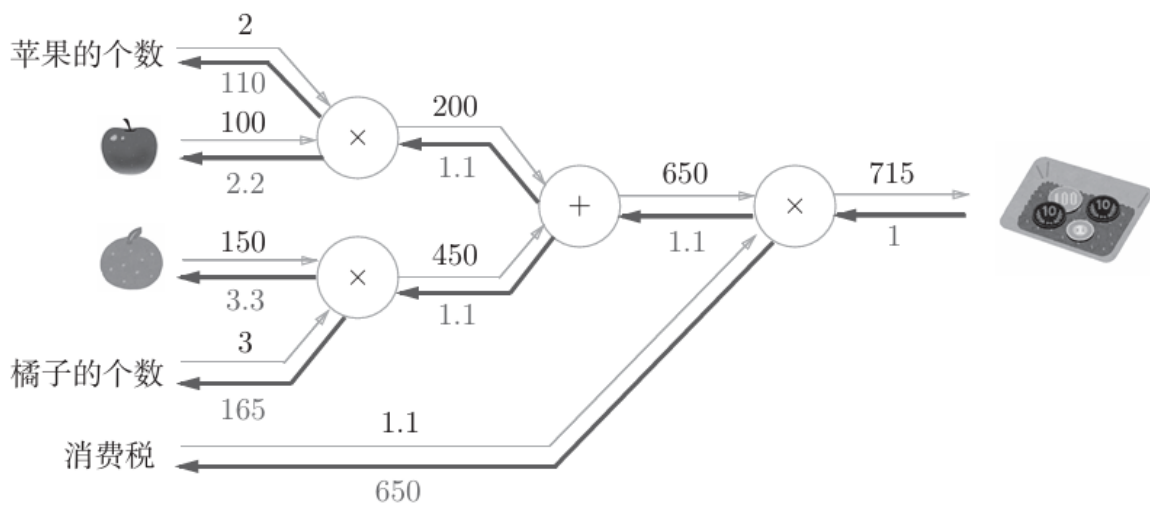
```
1 class MulLayer:
2     def __init__(self):
3         self.x = None
4         self.y = None
5     def forward(self, x, y):
6         self.x = x
7         self.y = y
8         out = x * y
9         return out
10    def backward(self, dout):
11        dx = dout * self.y # 翻转x和y
12        dy = dout * self.x
13        return dx, dy
14
15    apple = 100
16    apple_num = 2
17    tax = 1.1
18
19    # layer
20    mul_apple_layer = MulLayer()
21    mul_tax_layer = MulLayer()
22
23    # forward
24    apple_price = mul_apple_layer.forward(apple, apple_num)
25    price = mul_tax_layer.forward(apple_price, tax)
26
27    print(price) # 220.0
28
29    # backward
30    dprice = 1
31    dapple_price, dtax = mul_tax_layer.backward(dprice)
32    dapple, dapple_num = mul_apple_layer.backward(dapple_price)
33
34    print(dapple, dapple_num, dtax) # 2.2 110.0 220.0
```

## 4.4.2 加法层的实现

加法节点的加法层，如下所示：

```
1 class AddLayer:
2     def __init__(self):
3         pass
4     def forward(self, x, y):
5         out = x + y
6         return out
7     def backward(self, dout):
8         dx = dout * 1
9         dy = dout * 1
10        return dx, dy
```

计算图如下：



相关示例代码：

```
1 apple = 100
2 apple_num = 2
3 orange = 150
4 orange_num = 3
5 tax = 1.1
6
7 # layer
8 mul_apple_layer = MulLayer()
9 mul_orange_layer = MulLayer()
10 add_apple_orange_layer = AddLayer()
11 mul_tax_layer = MulLayer()
12
13 # forward
14 apple_price = mul_apple_layer.forward(apple, apple_num) #(1)
15 orange_price = mul_orange_layer.forward(orange, orange_num) #(2)
16 all_price = add_apple_orange_layer.forward(apple_price, orange_price) #(3)
17 price = mul_tax_layer.forward(all_price, tax) #(4)
18
19 # backward
20 dprice = 1
21 dall_price, dtax = mul_tax_layer.backward(dprice) #(4)
```

```

22 dapple_price, dorange_price = add_apple_orange_layer.backward(dall_price) #
   (3)
23 dorange, dorange_num = mul_orange_layer.backward(dorange_price) #(2)
24 dapple, dapple_num = mul_apple_layer.backward(dapple_price) #(1)
25
26 print(price) # 715
27 print(dapple_num, dapple, dorange, dorange_num, dtax) # 110 2.2 3.3 165 650

```

## 4.5 激活函数层的实现

### 4.5.1 ReLU层

激活函数ReLU (Rectified Linear Unit) 由下式表示:

$$y = \begin{cases} x & (x > 0) \\ 0 & (x \leq 0) \end{cases}$$

求出y关于x的导数

$$\frac{\partial y}{\partial x} = \begin{cases} 1 & (x > 0) \\ 0 & (x \leq 0) \end{cases}$$

计算图如下:



ReLU层代码:

```

1 class Relu:
2     def __init__(self):
3         self.mask = None
4     def forward(self, x):
5         self.mask = (x <= 0)
6         out = x.copy()
7         out[self.mask] = 0
8         return out
9     def backward(self, dout):
10        dout[self.mask] = 0
11        dx = dout
12        return dx

```

relu类有实例变量mask。

这个变量mask是由True/False构成的NumPy数组

- 它会把正向传播时的输入x的元素中小于等于0的地方保存为True
- 其他地方（大于0的元素）保存为False。

```
1 import numpy as np
2 x = np.array([[1.0, -0.5], [-2.0, 3.0]])
3 print(x)
4 # 结果如下:
5 # [[ 1. -0.5]
6 # [-2.  3. ]]
7
8
9 mask = (x <= 0)
10 print(mask)
11 # 结果如下:
12 # [[False  True]
13 #  [ True False]]
```

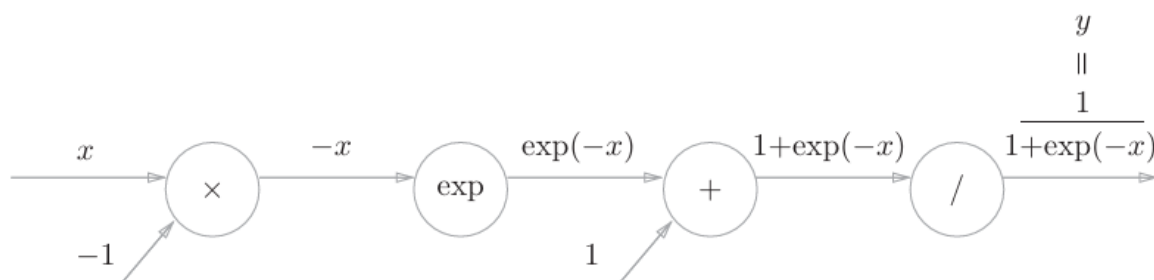
如果正向传播时的输入值小于等于0，则反向传播的值为0。因此，反向传播中会使用正向传播时保存的mask，将从上游传来的dout的 mask中的元素为True的地方设为0

## 4.5.2 Sigmoid层

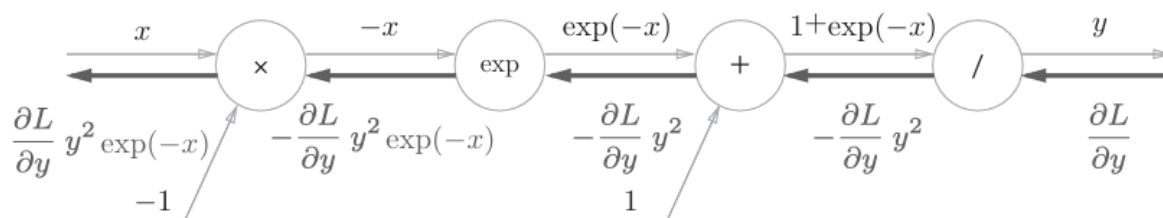
sigmoid函数：

$$h(x) = \frac{1}{1+e^{-x}}$$

计算图如下：（仅正向传播）



包含反向传播



示例代码：



```

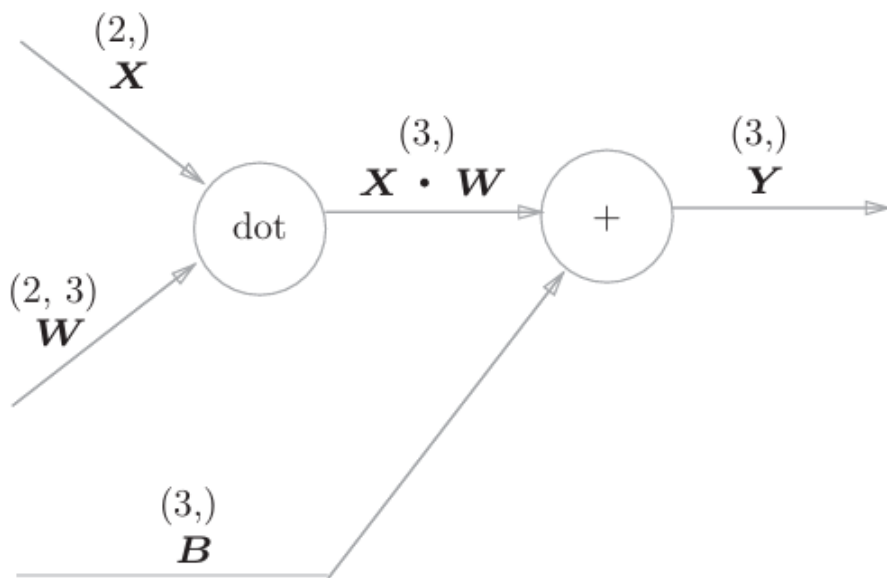
1 class Sigmoid:
2     def __init__(self):
3         self.out = None
4     def forward(self, x):
5         out = 1 / (1 + np.exp(-x))
6         self.out = out
7         return out
8     def backward(self, dout):
9         dx = dout * (1.0 - self.out) * self.out
10        return dx

```

## 4.6 Affine/Softmax层的实现

### 4.6.1 Affine层

计算图如下：

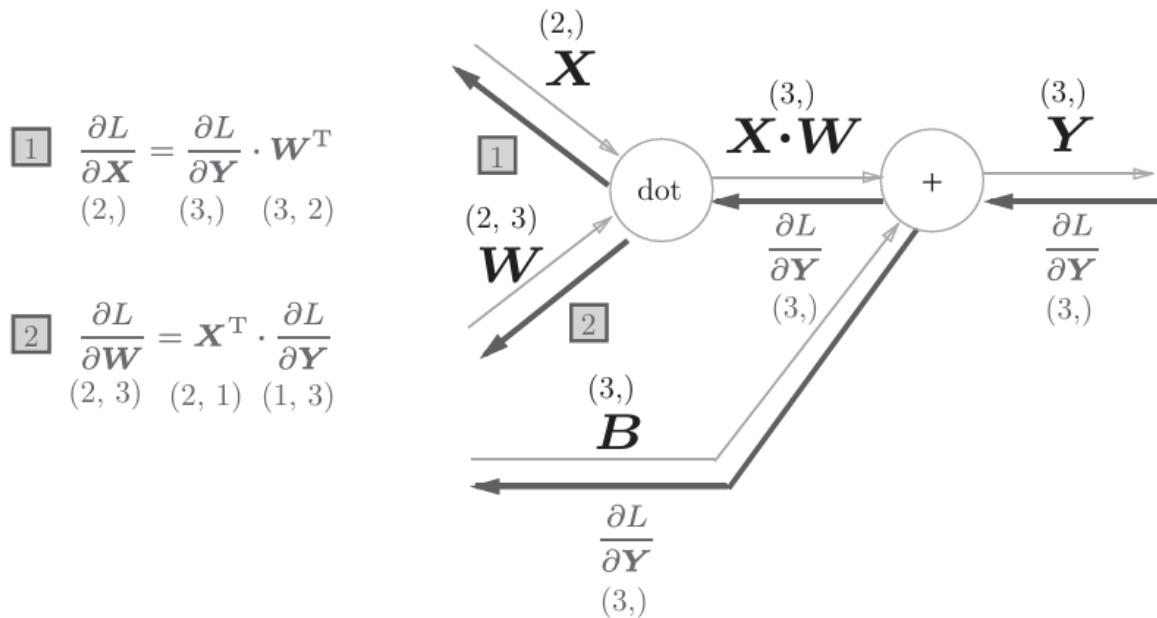


这个例子中各个节点间传播的是矩阵。

$$\frac{\partial L}{\partial \mathbf{X}} = \frac{\partial L}{\partial \mathbf{Y}} \cdot \mathbf{W}^T$$

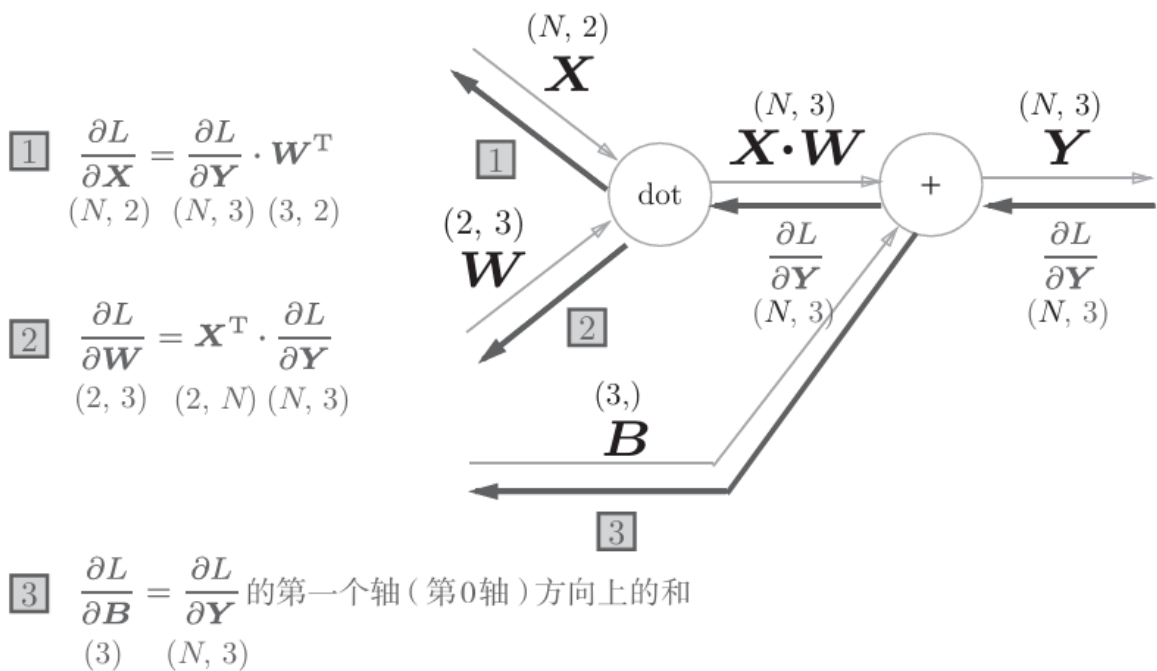
$$\frac{\partial L}{\partial \mathbf{W}} = \mathbf{X}^T \cdot \frac{\partial L}{\partial \mathbf{Y}}$$

计算图的反向传播



### 4.6.2 批版本的Affine层

考虑N 个数据一起进行正向传播的情况，也就是批版本的Affine层。



np.sum()对第0轴 (以数据为单位的轴, axis=0) 方向上的元素进行求和。

```
1 import numpy as np
2 dY = np.array([[1, 2, 3], [4, 5, 6]])
3 dB = np.sum(dY, axis=0) # axis=0表示对第0轴进行求和，即对每一列进行求和
4 print(dB) # [5 7 9]
```

Affine层的代码：

```
1 class Affine:
2     def __init__(self, w, b):
3         self.w = w
4         self.b = b
```

```

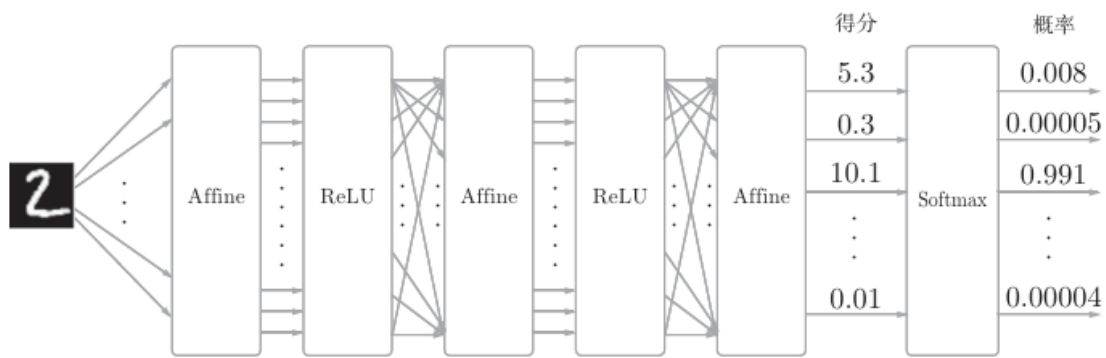
5         self.x = None
6         self.dw = None
7         self.db = None
8     def forward(self, x):
9         self.x = x
10        out = np.dot(x, self.w) + self.b
11        return out
12    def backward(self, dout):
13        dx = np.dot(dout, self.w.T)
14        self.dw = np.dot(self.x.T, dout)
15        self.db = np.sum(dout, axis=0)
16        return dx

```

### 4.6.3 Softmax-with-Loss层

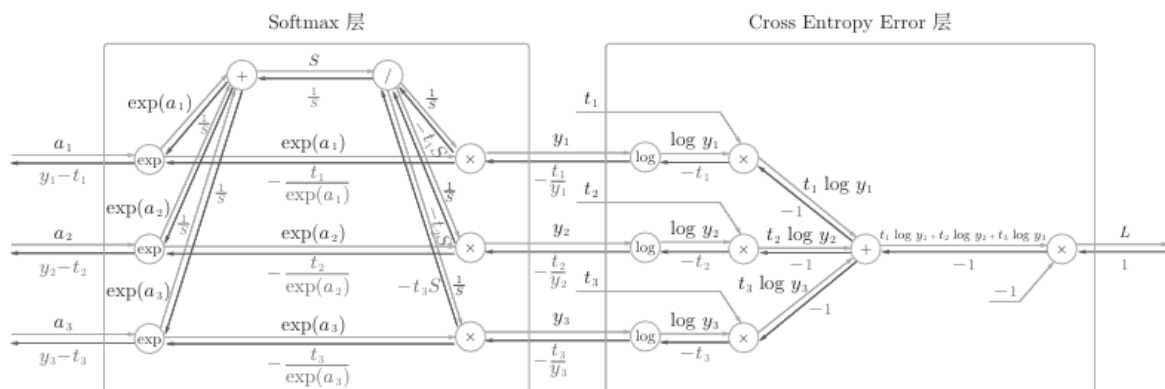
输出层的softmax函数。

比如手写数字识别时，Softmax层的输出如图所示

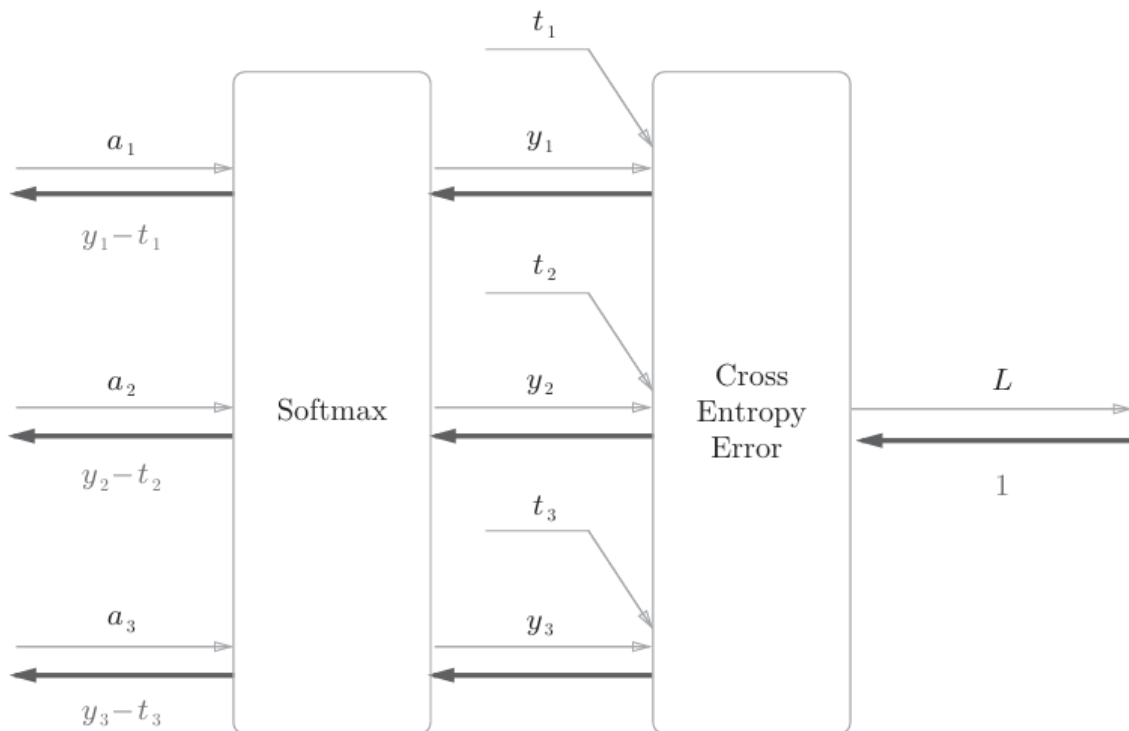


Softmax层将输入值正规化（将输出值的和调整为1）之后再输出。另外，因为手写数字识别要进行10类分类，所以向Softmax层的输入也有10个

Softmax-with-Loss层的计算图



简易版如下：



Softmax-with-Loss层的实现代码如下：

```

1  class SoftmaxWithLoss:
2      def __init__(self):
3          self.loss = None # 损失
4          self.y = None    # softmax的输出
5          self.t = None    # 监督数据 (one-hot vector)
6      def forward(self, x, t):
7          self.t = t
8          self.y = softmax(x)
9          self.loss = cross_entropy_error(self.y, self.t)
10         return self.loss
11     def backward(self, dout=1):
12         batch_size = self.t.shape[0]
13         dx = (self.y - self.t) / batch_size
14         return dx

```

## 4.7 误差反向传播法的实现

### 4.7.1 神经网络学习的全貌图

神经网络学习的步骤如下所示。

前提

- 神经网络中有合适的权重和偏置，调整权重和偏置以便拟合训练数据的
- 过程称为学习。神经网络的学习分为下面4个步骤。

步骤1 (mini-batch)

- 从训练数据中随机选择一部分数据。

步骤2 (计算梯度)

- 计算损失函数关于各个权重参数的梯度。

### 步骤3（更新参数）

- 将权重参数沿梯度方向进行微小的更新。

### 步骤4（重复）

- 重复步骤1、步骤2、步骤3。

之前介绍的误差反向传播法会在步骤2中出现。

## 4.7.2 对应误差反向传播法的神经网络的实现

### (1) TwolayerNet类中的实例变量

params：保存神经网络的参数的字典变量（实例变量）

- params['W1']是第1层的权重，params['b1']是第1层的偏置。
- params['W2']是第2层的权重，params['b2']是第2层的偏置

layers：保存神经网络的层的有序字典变量。

- 以layers['Affine1']、layers['ReLU1']、layers['Affine2']的形式，通过有序字典保存各个层

lastLayer：神经网络的最后一层。本例中为SoftmaxWithLoss层

### (2) TwoLayerNet类的方法

\_\_init\_\_(self, input\_size, hidden\_size, output\_size, weight\_init\_std)：进行初始化

- 参数从头开始依次是输入层的神经元数、隐藏层的 神经元数、输出层的神经元数、初始化权重时的高斯分布的规模

predict(self, x)：进行识别（推理）。

- 参数x是图像数据

loss(self, x, t)：计算损失函数的值。

- 参数 x是图像数据
- t是正确解标签（后面3个方法的参数也一样）

| accuracy(self, x, t)           | 计算识别精度                                     |
|--------------------------------|--------------------------------------------|
| numerical_gradient(self, x, t) | 计算权重参数的梯度                                  |
| gradient(self, x, t)           | 计算权重参数的梯度。numerical_gradient()的高速版，将在下一章实现 |

TwoLayerNet的代码实现

```
1 import sys, os
2 sys.path.append(os.pardir)
3 import numpy as np
4 from common.layers import *
5 from common.gradient import numerical_gradient
6 from collections import OrderedDict
7 class TwoLayerNet:
```

```

8     def __init__(self, input_size, hidden_size, output_size,
9                   weight_init_std=0.01):
10         #
11         初始化权重
12         self.params = {}
13         self.params['w1'] = weight_init_std * \
14             np.random.randn(input_size, hidden_size)
15         self.params['b1'] = np.zeros(hidden_size)
16         self.params['w2'] = weight_init_std * \
17             np.random.randn(hidden_size, output_size)
18         self.params['b2'] = np.zeros(output_size)
19         #
20         生成层
21         self.layers = OrderedDict()
22         self.layers['Affine1'] = \
23             Affine(self.params['w1'], self.params['b1'])
24         self.layers['Relu1'] = Relu()
25         self.layers['Affine2'] = \
26             Affine(self.params['w2'], self.params['b2'])
27         self.lastLayer = SoftmaxWithLoss()
28     def predict(self, x):
29         for layer in self.layers.values():
30             x = layer.forward(x)
31         return x
32     # x: 输入数据, t: 监督数据
33     def loss(self, x, t):
34         y = self.predict(x)
35         return self.lastLayer.forward(y, t)
36     def accuracy(self, x, t):
37         y = self.predict(x)
38         y = np.argmax(y, axis=1)
39         if t.ndim != 1 : t = np.argmax(t, axis=1)
40         accuracy = np.sum(y == t) / float(x.shape[0])
41         return accuracy
42     # x: 输入数据, t: 监督数据
43     def numerical_gradient(self, x, t):
44         loss_w = lambda w: self.loss(x, t)
45         grads = {}
46         grads['w1'] = numerical_gradient(loss_w, self.params['w1'])
47         grads['b1'] = numerical_gradient(loss_w, self.params['b1'])
48         grads['w2'] = numerical_gradient(loss_w, self.params['w2'])
49         grads['b2'] = numerical_gradient(loss_w, self.params['b2'])
50         return grads
51     def gradient(self, x, t):
52         # forward
53         self.loss(x, t)
54         # backward
55         dout = 1
56         dout = self.lastLayer.backward(dout)
57         layers = list(self.layers.values())
58         layers.reverse()
59         for layer in layers:
60             dout = layer.backward(dout)
61         设定
62         #
63         grads = {}
64         grads['w1'] = self.layers['Affine1'].dw
65         grads['b1'] = self.layers['Affine1'].db

```

```

66         grads['w2'] = self.layers['Affine2'].dw
67         grads['b2'] = self.layers['Affine2'].db
68         return grads
69

```

将神经网络的层保存为 OrderedDict 这一点非常重要。OrderedDict 是有序字典，“有序”是指它可以记住向字典里添加元素的顺序。

### 4.7.3 误差反向传播法的梯度确认

两种求梯度的方法

- 基于数值微分的方法
- 解析性地求解数学式的方法
  - 此方法通过使用误差反向传播法，即使存在大量的参数，也可以高效地计算梯度。

他俩优缺点：

- 数值微分的优点是实现简单，因此，一般情况下不太容易出错。
- 而误差反向传播法的实现很复杂，容易出错。

实际上，在确认误差反向传播法的实现是否正确时，是需要用到数值微分的。

确认数值微分求出的梯度结果和误差反向传播法求出的结果是否一致（严格地讲，是非常相近）的操作称为**梯度确认**（gradient check）。

梯度确认的代码实现如下所示

```

1  import sys, os
2  sys.path.append(os.pardir)
3  import numpy as np
4  from dataset.mnist import load_mnist
5  from two_layer_net import TwoLayerNet
6  # 读入数据
7  (x_train, t_train), (x_test, t_test) = \ load_mnist(normalize=True, one_
8  hot_label = True)
9  network = TwoLayerNet(input_size=784, hidden_size=50, output_size=10)
10 x_batch = x_train[:3]
11 t_batch = t_train[:3]
12 grad_numerical = network.numerical_gradient(x_batch, t_batch)
13 grad_backprop = network.gradient(x_batch, t_batch)
14 # 求各个权重的绝对误差的平均值
15 for key in grad_numerical.keys():
16     diff = np.average( np.abs(grad_backprop[key] - grad_numerical[key]) )
17     print(key + ":" + str(diff))

```

### 4.7.4 使用误差反向传播法的学习

使用了误差反向传播法的神经网络的学习的实现

代码如下：

```

1  # =====
2  # 1. 环境准备
3  # =====
4  import sys, os
5  # 将父目录加入 Python 路径，以便导入自定义包（如 dataset、two_layer_net）

```

```

6 sys.path.append(os.pardir)
7
8 import numpy as np
9 # 导入自己写的 MNIST 加载接口
10 from dataset.mnist import load_mnist
11 # 导入自己写的两层全连接神经网络类
12 from two_layer_net import TwoLayerNet
13
14 # =====
15 # 2. 读取 MNIST 数据
16 # =====
17 # normalize=True : 将像素值缩放到 0.0~1.0
18 # one_hot_label=True : 标签变成 one-hot 向量
19 (x_train, t_train), (x_test, t_test) = \
20     load_mnist(normalize=True, one_hot_label=True)
21
22 # =====
23 # 3. 初始化网络与超参数
24 # =====
25 # 784 输入神经元 (28x28 像素)
26 # 50 隐藏层神经元
27 # 10 输出神经元 (0~9 十个类别)
28 network = TwoLayerNet(input_size=784, hidden_size=50, output_size=10)
29
30 iters_num = 10000 # 总训练迭代次数
31 train_size = x_train.shape[0] # 训练集样本数 (60000)
32 batch_size = 100 # 每次小批量随机梯度下降的样本数
33 learning_rate = 0.1 # 学习率
34 train_loss_list = [] # 记录每轮迭代的损失
35 train_acc_list = [] # 记录训练集精度
36 test_acc_list = [] # 记录测试集精度
37
38 # 1 个 epoch 需要多少次迭代 (600 次)
39 iter_per_epoch = max(train_size / batch_size, 1)
40
41 # =====
42 # 4. 训练主循环
43 # =====
44 for i in range(iters_num):
45     # 4.1 随机挑选 batch_size 张图片
46     batch_mask = np.random.choice(train_size, batch_size)
47     x_batch = x_train[batch_mask]
48     t_batch = t_train[batch_mask]
49
50     # 4.2 通过误差反向传播求梯度
51     grad = network.gradient(x_batch, t_batch)
52
53     # 4.3 梯度下降更新参数
54     for key in ('w1', 'b1', 'w2', 'b2'):
55         network.params[key] -= learning_rate * grad[key]
56
57     # 4.4 计算当前小批量的损失并记录
58     loss = network.loss(x_batch, t_batch)
59     train_loss_list.append(loss)
60
61     # 4.5 每个 epoch 结束, 计算并打印一次精度
62     if i % iter_per_epoch == 0:
63         train_acc = network.accuracy(x_train, t_train)

```



```
64     test_acc = network.accuracy(x_test, t_test)
65     train_acc_list.append(train_acc)
66     test_acc_list.append(test_acc)
67     print(train_acc, test_acc)
```

---

# 第5章 与学习相关的技巧

本章将介绍神经网络的学习中的一些重要观点，主题涉及寻找最优权重参数的最优化方法、权重参数的初始值、超参数的设定方法等。

## 5.1 参数的更新

神经网络的学习的目的是找到使损失函数的值尽可能小的参数。这是寻找最优参数的问题，解决问题的过程称为**最优化**（optimization）

使用参数的梯度，沿梯度方向更新参数，并重复这个步骤多次，从而逐渐靠近最优参数，这个过程称为**随机梯度下降法**（stochastic gradient descent），简称**SGD**

### 5.1.1 SGD

$$W \leftarrow W - \eta \frac{\partial L}{\partial W}$$

- $W$ : 权重参数
- $\frac{\partial L}{\partial W}$ : 损失函数关于 $W$ 的梯度
- $\eta$ : 表示学习率

SGD实现代码：

```
1 class SGD:
2     def __init__(self, lr=0.01):
3         # lr: 学习率
4         self.lr = lr
5     def update(self, params, grads):
6         for key in params.keys():
7             params[key] -= self.lr * grads[key]
```

#### SGD的缺点

SGD低效的根本原因是，梯度的方向并没有指向最小值的方向。

为了改正SGD的缺点，下面我们将介绍Momentum、AdaGrad、Adam这3种方法来取代SGD。

### 5.1.2 Momentum

Momentum是“动量”的意思，和物理有关。

数学表达如下：

$$v \leftarrow \alpha v - \eta \frac{\partial L}{\partial W}$$

$$W \leftarrow W + v$$

- $W$ : 权重参数

- $\frac{\partial L}{\partial W}$ : 损失函数关于W的梯度
- $\eta$ : 表示学习率
- v: 速度

公式表示了物体在梯度方向上受力，在这个力的作用下，物体的速度增加这一物理法则。

Momentum的代码实现：

```
1 class Momentum:
2     def __init__(self, lr=0.01, momentum=0.9):
3         self.lr = lr
4         self.momentum = momentum
5         self.v = None
6     def update(self, params, grads):
7         if self.v is None:
8             self.v = {}
9             for key, val in params.items():
10                 self.v[key] = np.zeros_like(val)
11         for key in params.keys():
12             self.v[key] = self.momentum*self.v[key] - self.lr*grads[key]
13             params[key] += self.v[key]
```

### 5.1.3 AdaGrad

在神经网络的学习中

学习率（数学式中记为 $\eta$ ）的值很重要。

- 学习率过小，会导致学习花费过多时间；
- 学习率过大，则会导致学习发散而不能正确进行。

**学习率衰减** (learning rate decay)：即随着学习的进行，使学习率逐渐减小。

数学式表示AdaGrad的更新方法：

$$\mathbf{h} \leftarrow \mathbf{h} + \frac{\partial L}{\partial \mathbf{W}} \odot \frac{\partial L}{\partial \mathbf{W}}$$

$$\mathbf{W} \leftarrow \mathbf{W} - \eta \frac{1}{\sqrt{\mathbf{h}}} \frac{\partial L}{\partial \mathbf{W}}$$

AdaGrad会记录过去所有梯度的平方和。因此，学习越深入，更新的幅度就越小。

AdaGrad的实现代码：

```
1 class AdaGrad:
2
3     """AdaGrad"""
4
5     def __init__(self, lr=0.01):
6         self.lr = lr
7         self.h = None
8
```

```

9     def update(self, params, grads):
10         if self.h is None:
11             self.h = {}
12         for key, val in params.items():
13             self.h[key] = np.zeros_like(val)
14
15         for key in params.keys():
16             self.h[key] += grads[key] * grads[key]
17             params[key] -= self.lr * grads[key] / (np.sqrt(self.h[key]) +
1e-7)

```

这里需要注意的是，最后一行加上了微小值 $1e-7$ 。这是为了防止当 `self.h[key]` 中有0时，将0用作除数的情况。

## 5.1.4 Adam

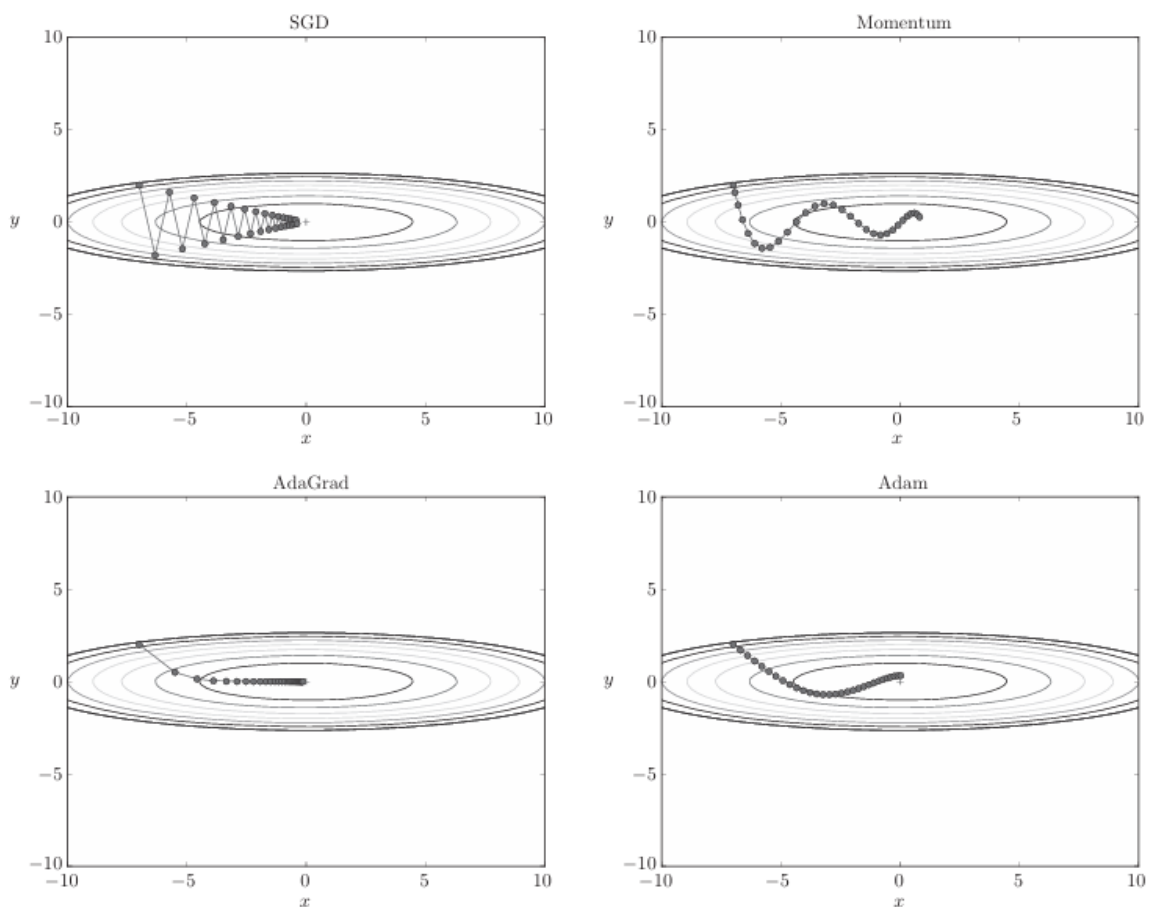
Adam：Momentum参照小球在碗中滚动的物理规则进行移动，AdaGrad为参数的每个元素适当地调整更新步伐。将这两个方法融合在一起就是Adam方法的基本思路

Adam会设置3个超参数。

- 一个是学习率（论文中以 $\alpha$ 出现）
- 另外两个是一次momentum系数 $\beta_1$
- 二次momentum系数 $\beta_2$

根据论文，标准的设定值是 $\beta_1$ 为0.9， $\beta_2$ 为0.999。设置了这些值后，大多数情况下都能顺利运行。

4种更新参数的方法对比：



## 5.2 权重的初始值

## 5.2.1 可以将权重初始值设为0吗

**权值衰减**就是一种以减小权重参数的值为目的进行学习的方法。通过减小权重参数的值来抑制过拟合的发生。

如果想减小权重的值，一开始就将初始值设为较小的值才是正途。

- 实际上，在这之前的权重初始值都是像`0.01 * np.random.randn(10, 100)`这样，使用 由高斯分布生成的值乘以0.01后得到的值（标准差为0.01的高斯分布）。

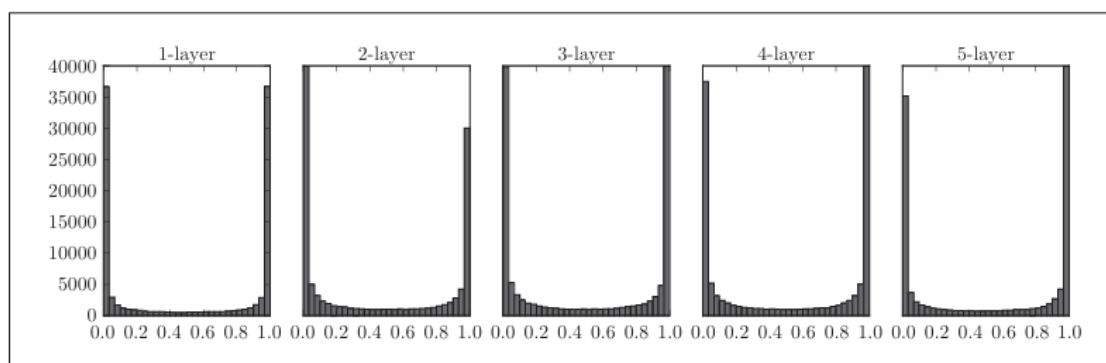
将权重初始值设为 0的话，将无法正确进行学习。

## 5.2.2 隐藏层的激活值的分布

因此，偏向0和1的数据分布会造成反向传播中梯度的值不断变小，最后消失。这个问题称为梯度消失。各层的激活值的分布都要求有适当的广度。

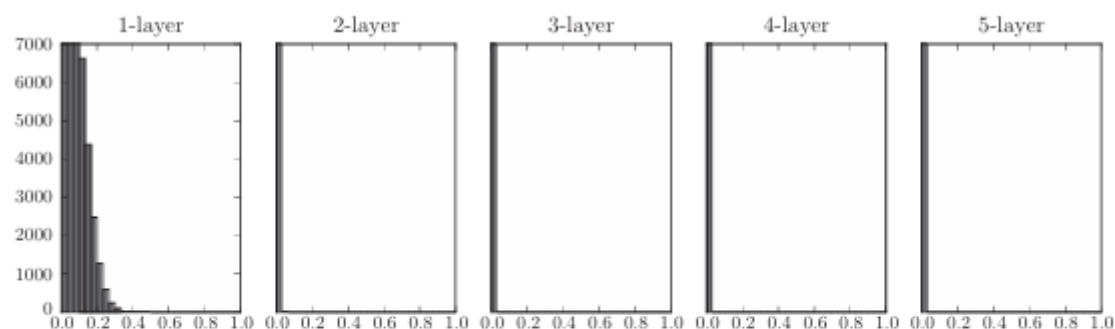
用作激活函数的函数最好具有关于原点对称的性质。

```
1 import numpy as np
2 import matplotlib.pyplot as plt
3 def sigmoid(x):
4     return 1 / (1 + np.exp(-x))
5 x = np.random.randn(1000, 100) # 1000个数据
6 node_num = 100 # 各隐藏层的节点（神经元）数
7 hidden_layer_size = 5 # 隐藏层有5层
8 activations = {} # 激活值的结果保存在这里
9 for i in range(hidden_layer_size):
10     if i != 0:
11         x = activations[i-1]
12         w = np.random.randn(node_num, node_num) * 1
13         z = np.dot(x, w)
14         a = sigmoid(z) # sigmoid函数
15         activations[i] = a
16 # 绘制直方图
17 for i, a in activations.items():
18     plt.subplot(1, len(activations), i+1)
19     plt.title(str(i+1) + "-layer")
20     plt.hist(a.flatten(), 30, range=(0,1))
21 plt.show()
```

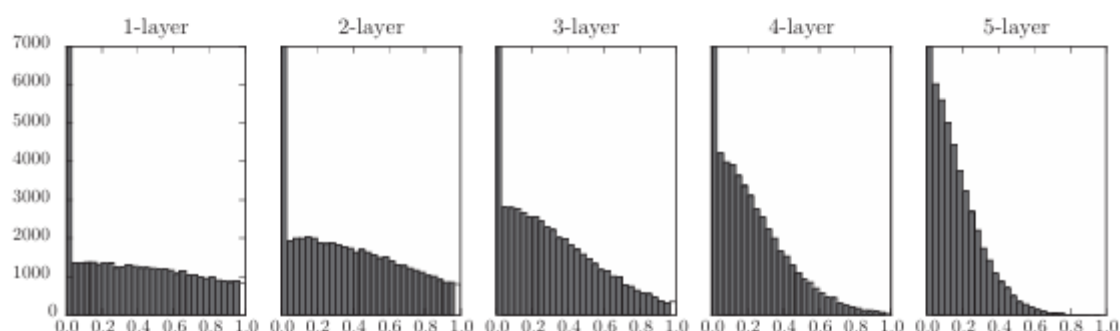


使用标准差为1的高斯分布作为权重初始值时的各层激活值的分布

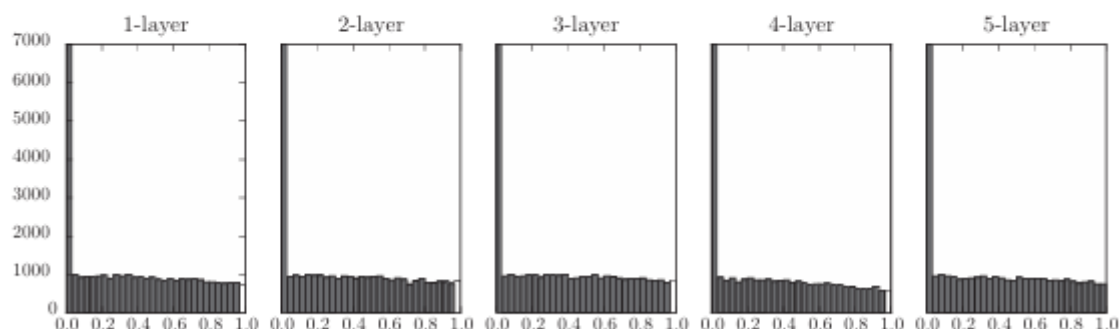
## 5.2.3 ReLU的权重初始值



权重初始值为标准差是0.01的高斯分布时



权重初始值为 Xavier 初始值时



权重初始值为 He 初始值时

观察实验结果可知，当“std=0.01”时，各层的激活值非常小。神经网络上传递的是非常小的值，说明逆向传播时权重的梯度也同样很小。这是很严重的问题，实际上学习基本上没有进展。

总结一下，

- 当激活函数使用ReLU时，权重初始值使用He初始值
- 当激活函数为sigmoid或tanh等S型曲线函数时，初始值使用Xavier初始值。这是目前的最佳实践。

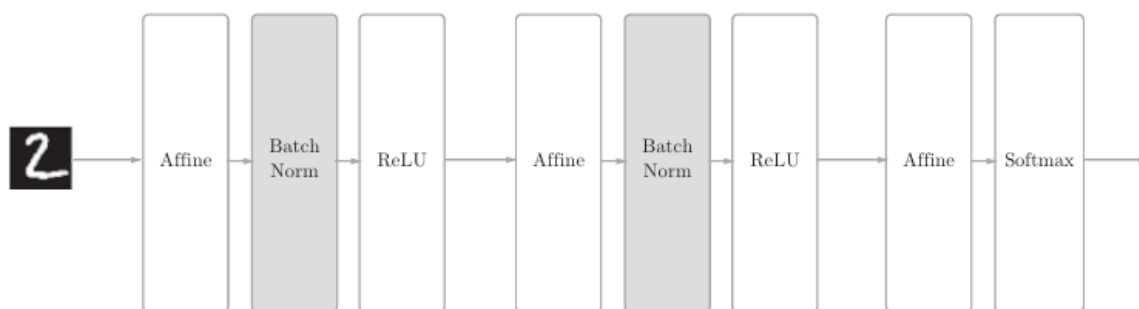
## 5.3 Batch Normalization

### 5.3.1 Batch Normalization算法

Batch Norm有以下优点：

- 可以使学习快速进行（可以增大学习率）
- 不那么依赖初始值（对于初始值不用那么神经质）
- 抑制过拟合（降低Dropout等的必要性）。

使用了Batch Normalization的神经网络的例子（Batch Norm层的背景为灰色）



Batch Norm，顾名思义，以进行学习时的mini-batch为单位，按mini batch进行正规化。具体而言，就是进行使数据分布的均值为0、方差为1的正规化。用数学式表示的话，如下所示。

$$\mu_B \leftarrow \frac{1}{m} \sum_{i=1}^m x_i$$
$$\sigma_B^2 \leftarrow \frac{1}{m} \sum_{i=1}^m (x_i - \mu_B)^2$$
$$\hat{x}_i \leftarrow \frac{x_i - \mu_B}{\sqrt{\sigma_B^2 + \epsilon}}$$

## 5.4 正则化

**过拟合**是一个很常见的问题。过拟合指的是只能拟合训练数据，但不能很好地拟合不包含在训练数据中的其他数据的状态。

### 5.4.1 过拟合

发生过拟合的原因（两个）

- 模型拥有大量参数、表现力强。
- 训练数据少。

### 5.4.2 权值衰减

**权值衰减**是一直以来经常被使用的一种抑制过拟合的方法。该方法通过在学习的过程中对大的权重进行惩罚，来抑制过拟合。很多过拟合原本就是因为权重参数取值过大才发生的。

### 5.4.3 Dropout

**Dropout**是一种在学习的过程中随机删除神经元的方法。

训练时，随机选出隐藏层的神经元，然后将其删除。

被删除的神经元不再进行信号的传递。

训练时，每传递一次数据，就会随机选择要删除的神经元。然后，测试时，虽然会传递所有的神经元信号，但是对于各个神经元的输出，要乘上训练时的删除比例后再输出。

Dropout实现代码：

```
1 class Dropout:
2     def __init__(self, dropout_ratio=0.5):
3         self.dropout_ratio = dropout_ratio
4         self.mask = None
5     def forward(self, x, train_flg=True):
6         if train_flg:
7             self.mask = np.random.rand(*x.shape) > self.dropout_ratio
8             return x * self.mask
9         else:
10            return x * (1.0 - self.dropout_ratio)
11    def backward(self, dout):
12        return dout * self.mask
```

每次正向传播时

- self.mask中都会以False的形式保存要删除的神经元
- self.mask会随机生成和x形状相同的数组，并将值比 dropout\_ratio大的元素设为True。

反向传播时的行为和ReLU相同。

## 5.5 超参数的验证

**超参数**是指，比如各层的神经元数量、batch大小、参数更新时的学习率或权值衰减等

### 5.5.1 验证数据

之前我们使用的数据集分成了训练数据和测试数据，训练数据用于学习，测试数据用于评估泛化能力。

这里要注意的是，不能使用测试数据评估超参数的性能。这一点非常重要，但也容易被忽视。

为什么不能用测试数据评估超参数的性能呢？

- 这是因为如果使用测试数据调整超参数，超参数的值会对测试数据发生过拟合。

用于调整超参数的数据，一般称为**验证数据**（validation data）。我们使用这个验证数据来评估超参数的好坏。

### 5.5.2 超参数的最优化

进行超参数的最优化时，逐渐缩小超参数的“好值”的存在范围非常重要。

超参数最优化步骤如下：

步骤0

- 设定超参数的范围。

步骤1

- 从设定的超参数范围中随机采样。

步骤2

- 使用步骤1中采样到的超参数的值进行学习，通过验证数据评估识别精度（但是要将epoch设置得很小）。

步骤3

- 重复步骤1和步骤2（100次等），根据它们的识别精度的结果，缩小超参数的范围。



在超参数的最优化中，如果需要更精炼的方法，可以使用**贝叶斯最优化** (Bayesian optimization)。贝叶斯最优化运用以贝叶斯定理为中心的数学理论，能够更加严密、高效地进行最优化。

---

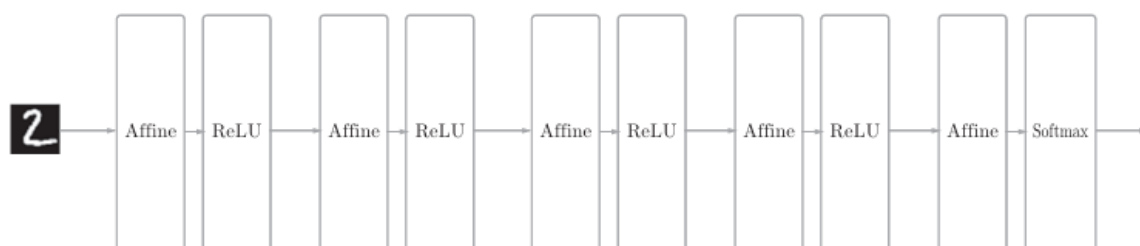
# 第6章 卷积神经网络(CNN)

CNN被用于图像识别、语音识别等各种场合，在图像识别的比赛中，基于深度学习的方法几乎都以CNN为基础。

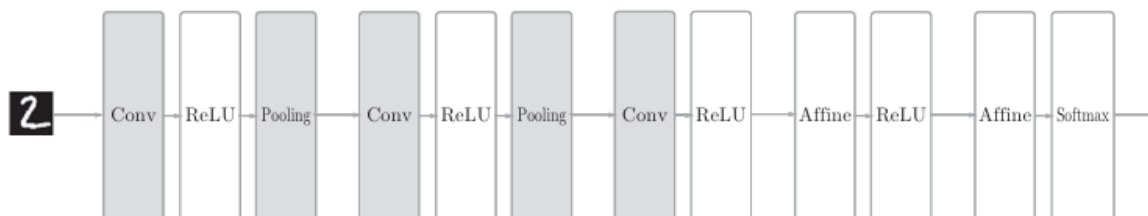
## 6.1 整体结构

**全连接** (fully-connected)：相邻层的所有神经元之间都有连接

全连接图的网络如下：



CNN的网络例子如下：



## 6.2 卷积层

### 6.2.1 全连接层存在的问题

#### (1) 全连接层问题

在全连接层（Affine层）中，相邻层的神经元全部连接在一起，输出的数量可以任意决定。

存在的问题：

- 数据的形状被“忽视”

因为全连接层会忽视形状，将全部的输入数据作为相同的神经元（同一维度的神经元）处理，所以无法利用与形状相关的信息。

#### (2) 卷积层优势

**卷积层**可以保持形状不变。

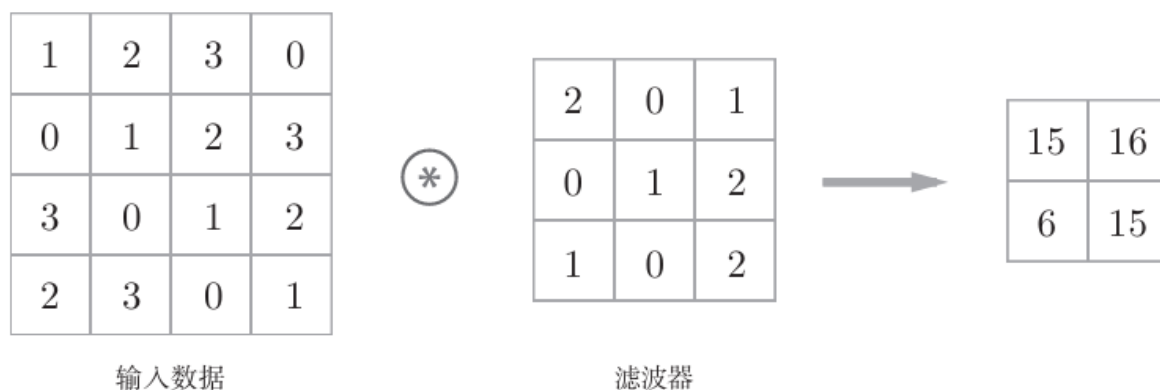
将卷积层的输入输出数据称为特征图（feature map）。

- 卷积层的输入数据称为输入特征图（input feature map）
- 输出数据称为输出特征图（output feature map）

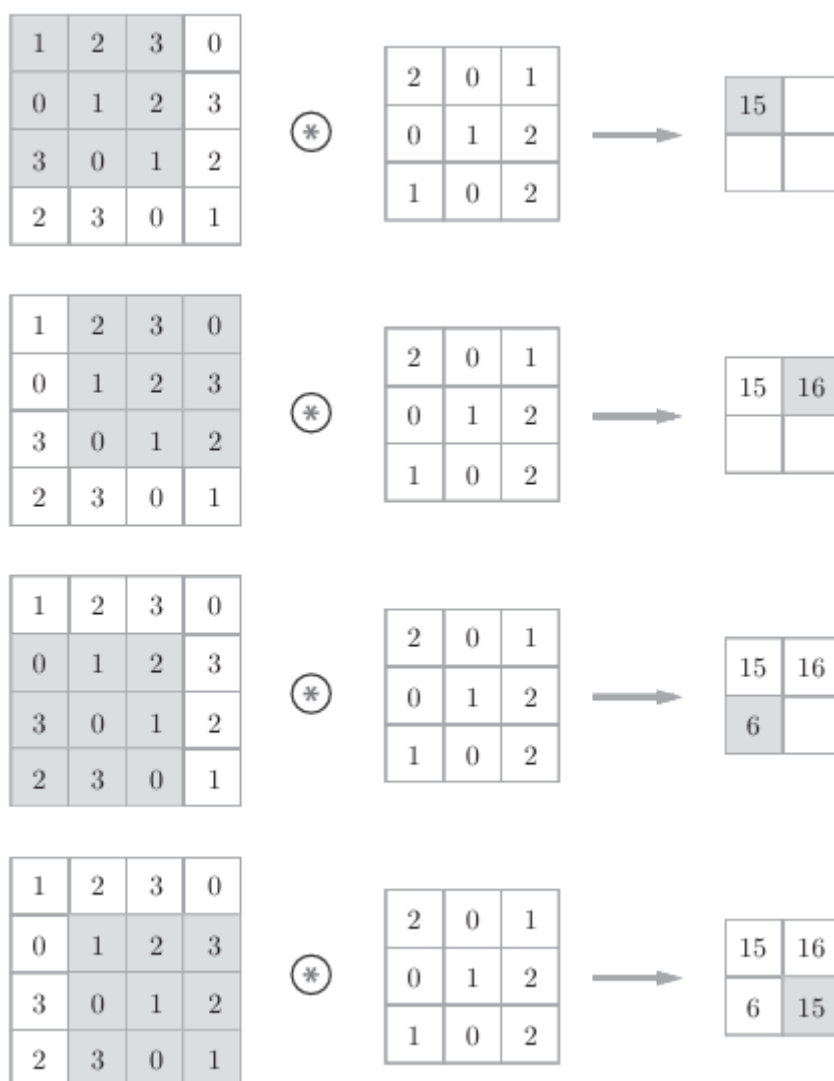
## 6.2.2 卷积运算

卷积层进行的处理就是卷积运算。卷积运算相当于图像处理中的“滤波器运算”

如下图所示：



这种计算过程是，滤波器中的矩阵形状与输入数据中的相同形状进行矩阵内积，从左上角开始，然后再往右一格进行下一次计算...



每次都是阴影部分和滤波器进行矩阵内积（相同位置相乘，然后再把所有数据相加）

### 6.2.3 填充

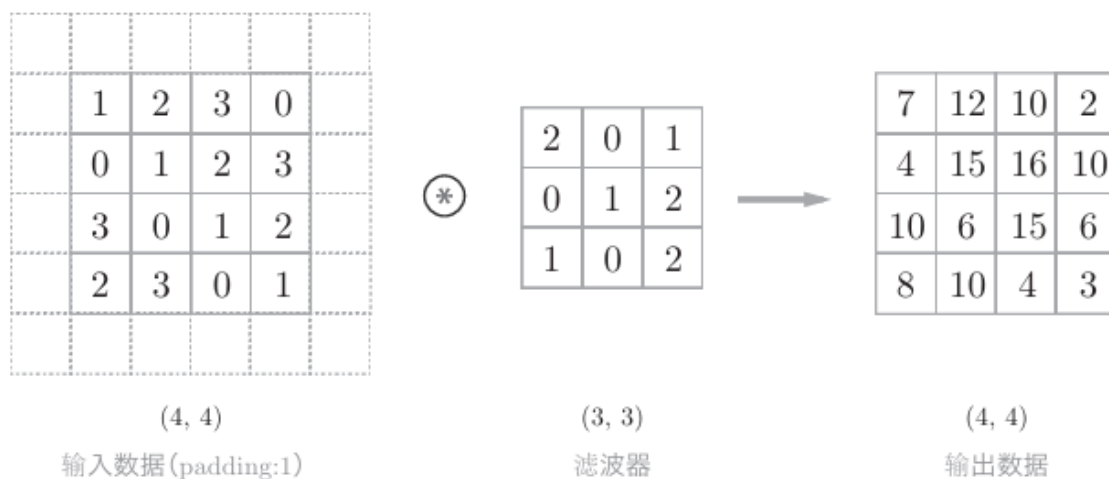
在进行卷积层的处理之前，有时要向输入数据的周围填入固定的数据（比如0等），这称为**填充**（padding），是卷积运算中经常会用到的处理。

使用填充主要是为了调整输出的大小。

比如，对大小为(4,4)的输入数据应用(3,3)的滤波器时，输出大小变为(2,2)，相当于输出大小比输入大小缩小了2个元素。为了保证输出依然为(4, 4)

因此，卷积运算就可以在保持空间大小不变的情况下将数据传给下一层。

如下图所示，向输入数据的周围填入0（虚线框）



### 6.2.4 步幅

应用滤波器的位置间隔称为步幅（stride）

步幅设置为2，如下图所示：

|   |   |   |   |   |   |   |
|---|---|---|---|---|---|---|
| 1 | 2 | 3 | 0 | 1 | 2 | 3 |
| 0 | 1 | 2 | 3 | 0 | 1 | 2 |
| 3 | 0 | 1 | 2 | 3 | 0 | 1 |
| 2 | 3 | 0 | 1 | 2 | 3 | 0 |
| 1 | 2 | 3 | 0 | 1 | 2 | 3 |
| 0 | 1 | 2 | 3 | 0 | 1 | 2 |
| 3 | 0 | 1 | 2 | 3 | 0 | 1 |

⊗

|   |   |   |
|---|---|---|
| 2 | 0 | 1 |
| 0 | 1 | 2 |
| 1 | 0 | 2 |



|    |  |  |
|----|--|--|
| 15 |  |  |
|    |  |  |
|    |  |  |

步幅: 2

|   |   |   |   |   |   |   |
|---|---|---|---|---|---|---|
| 1 | 2 | 3 | 0 | 1 | 2 | 3 |
| 0 | 1 | 2 | 3 | 0 | 1 | 2 |
| 3 | 0 | 1 | 2 | 3 | 0 | 1 |
| 2 | 3 | 0 | 1 | 2 | 3 | 0 |
| 1 | 2 | 3 | 0 | 1 | 2 | 3 |
| 0 | 1 | 2 | 3 | 0 | 1 | 2 |
| 3 | 0 | 1 | 2 | 3 | 0 | 1 |

⊗

|   |   |   |
|---|---|---|
| 2 | 0 | 1 |
| 0 | 1 | 2 |
| 1 | 0 | 2 |



|    |    |  |
|----|----|--|
| 15 | 17 |  |
|    |    |  |
|    |    |  |

综上,

- 增大步幅后, 输出大小会变小。
- 增大填充后, 输出大小会变大。

### 输出大小的计算方法

假设输入大小为(H,W), 滤波器大小为(FH,FW), 输出大小为 (OH,OW), 填充为P, 步幅为S。此时, 输出大小可通过下式进行计算。

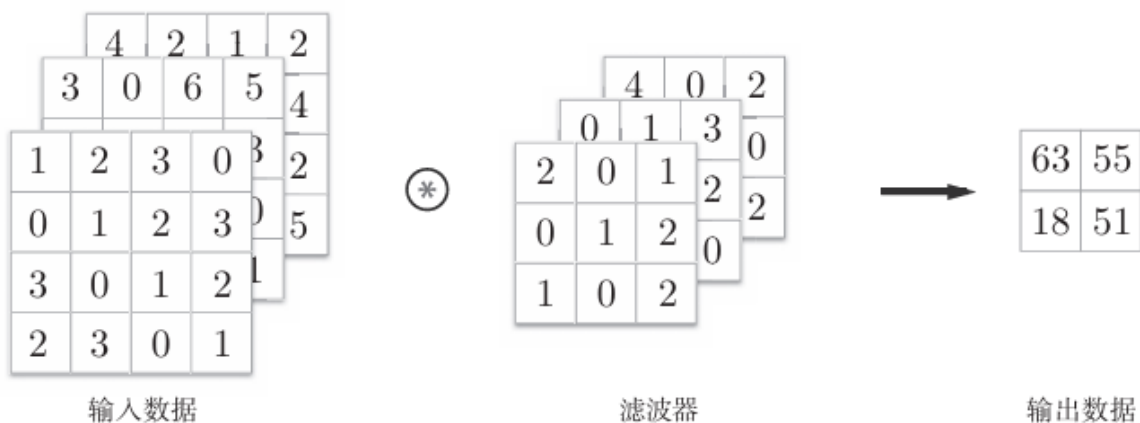
$$OH = \frac{H + 2P - FH}{S} + 1$$

$$OW = \frac{W + 2P - FW}{S} + 1$$

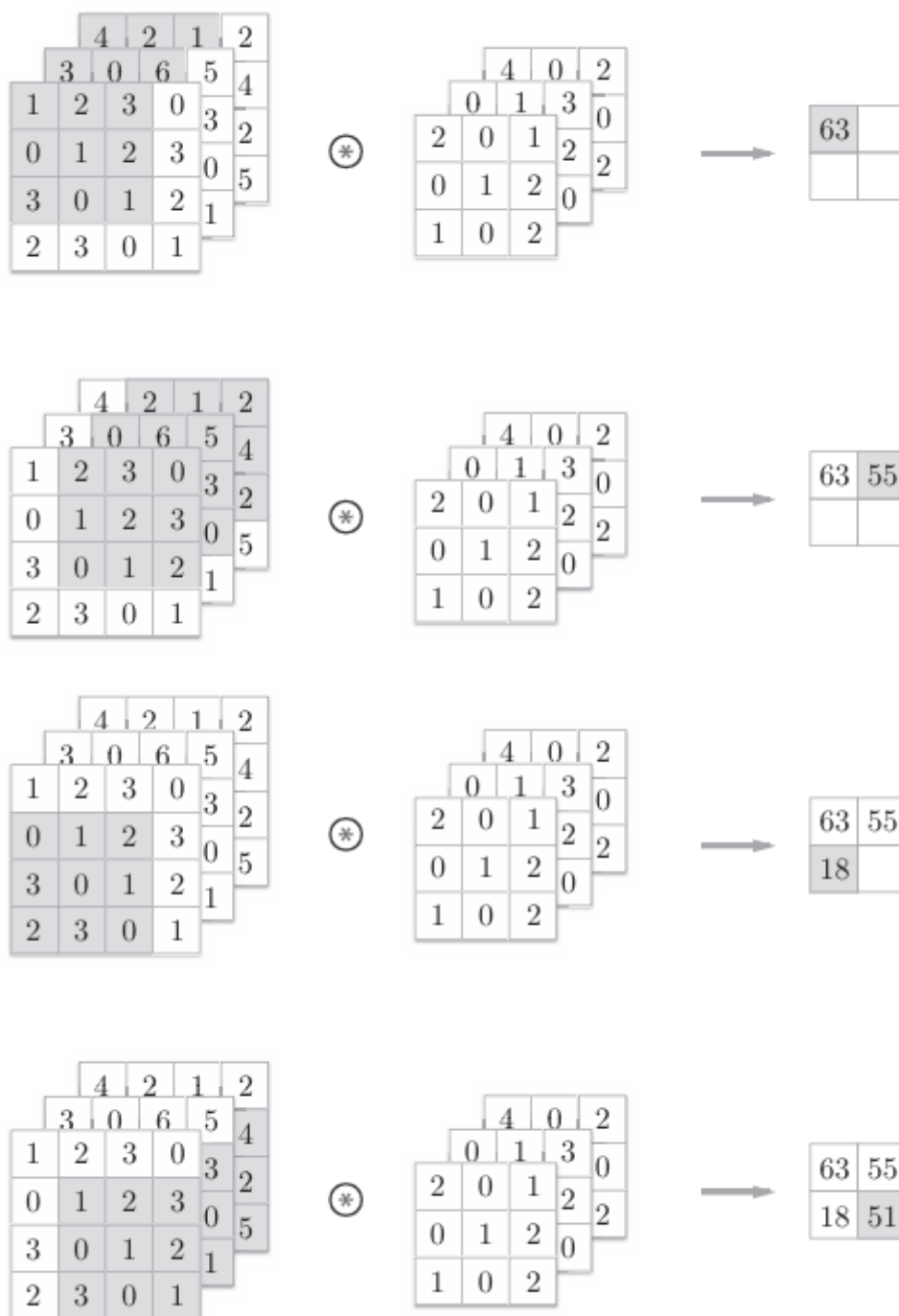
## 6.2.5 三维数据的卷积运算

之前的卷积运算的例子都是以有高、长方形的2维形状为对象的。但是, 图像是3维数据, 除了高、长方方向之外, 还需要处理通道方向。

如下图所示:



运算过程如下:



需要注意的是，在3维数据的卷积运算中，输入数据和滤波器的通道数 要设为相同的值。

## 6.2.6 结合方块思考

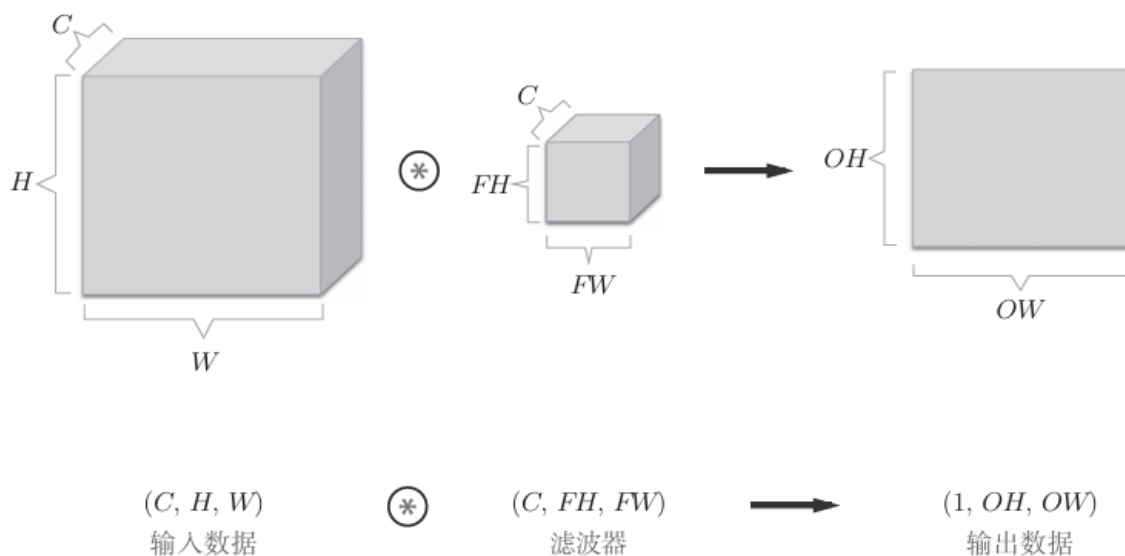
将数据和滤波器结合长方体的方块来考虑，3维数据的卷积运算会很容易理解。

方块是如下图所示的3维长方体。把3维数据表示为多维数组时，书写顺序为（channel, height, width）。

- 比如，通道数为C、高度为H、长度为W的数据
- 形状可以写成（C,H,W）。

滤波器也一样，要按（channel, height, width）的顺序书写。

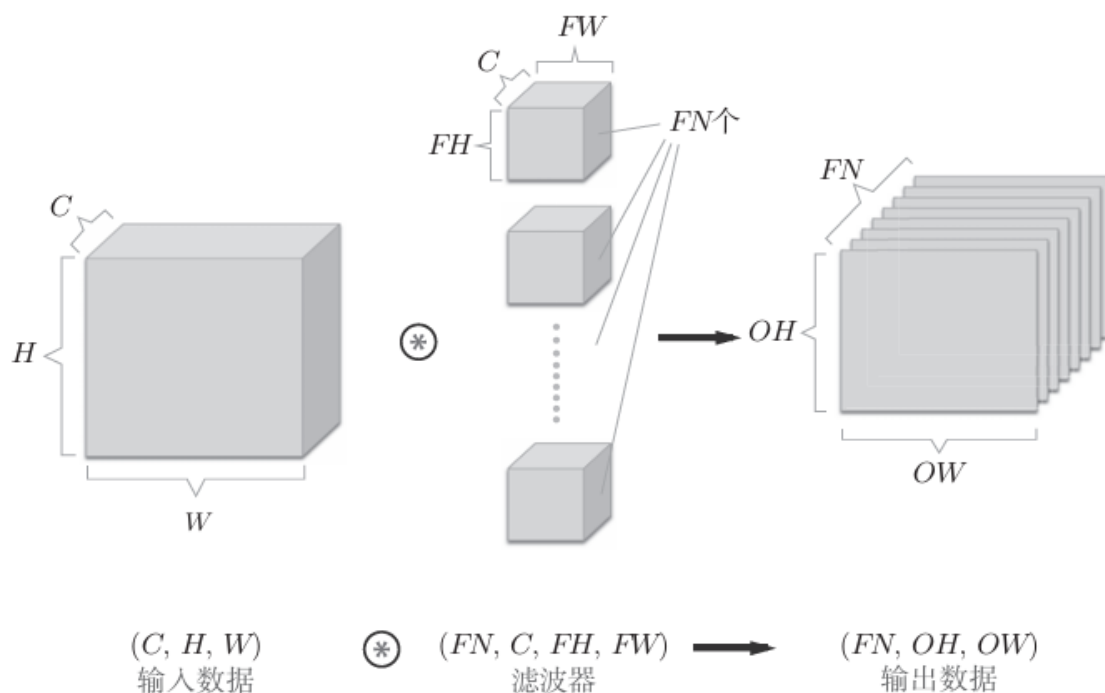
- 比如，通道数为C、滤波器高度为FH（Filter Height）、长度为FW（Filter Width）时
- 可以写成（C,FH,FW）。



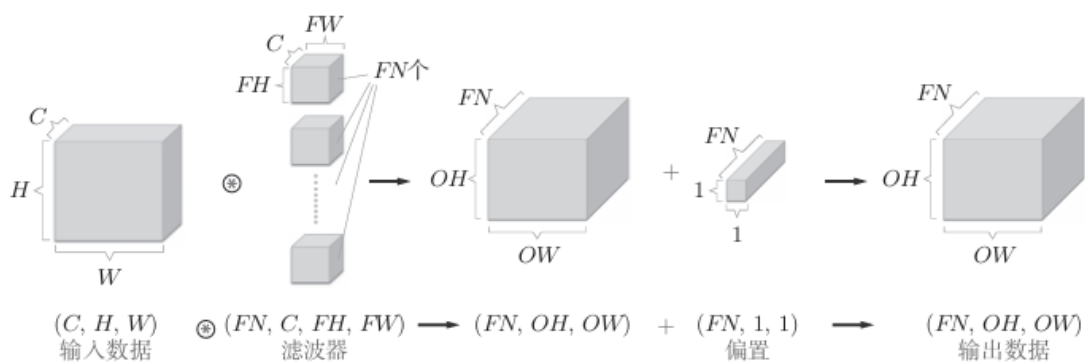
在这个例子中，数据输出是1张特征图。所谓1张特征图，换句话说，就是通道数为1的特征图。

如果要在通道方向上也拥有多个卷积运算的输出，该怎么做呢？为此，就需要用到多个滤波器（权重）。

如下图所示：



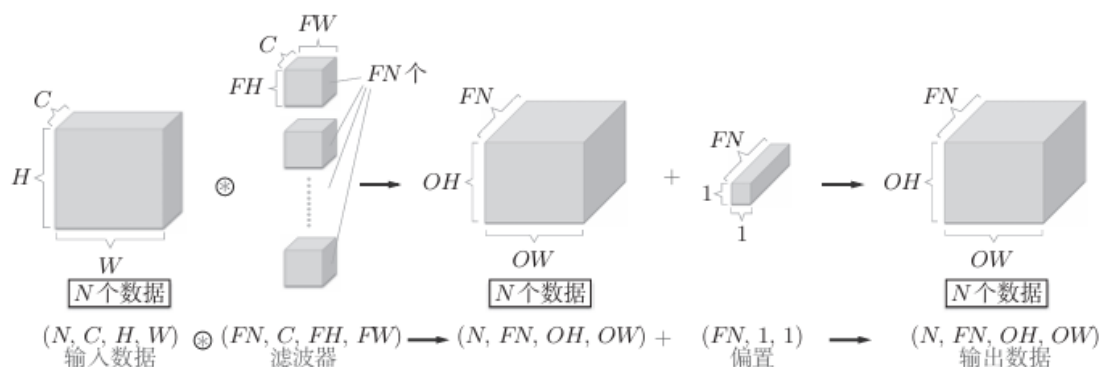
卷积运算中（和全连接层一样）存在偏置。如果进一步追加偏置的加法运算处理，则结果如下图所示。



## 6.2.7 批处理

将上面图中的处理改成对N个数据进行批处理时，数据的形状如下图所示。

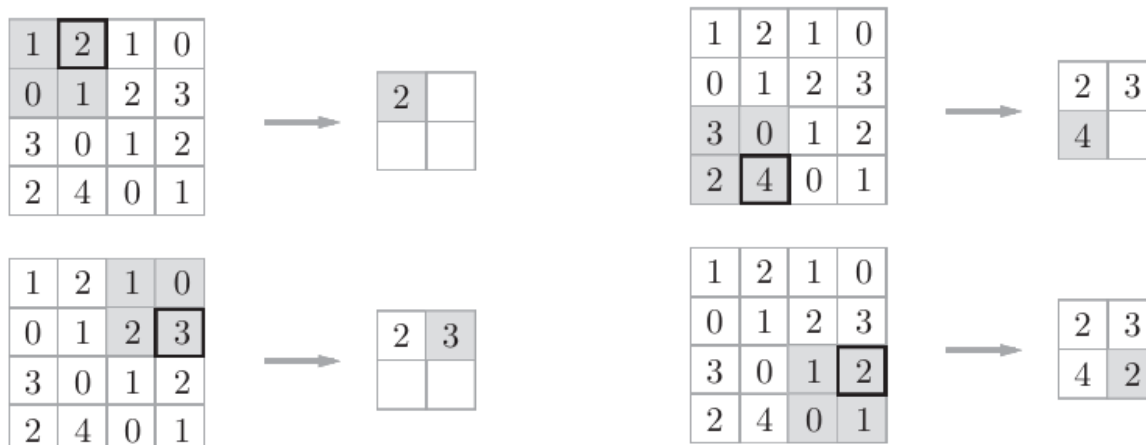
批处理版的数据流中，在各个数据的开头添加了批用的维度。



## 6.3 池化层

池化是缩小高、长方向上的空间的运算。

例如下图，将 $2 \times 2$ 的区域集约成1个元素的处理，缩小空间大小



上述图中每次选择区域中最大的元素（Max池化），并且将步幅设为2

- Max池化是从目标区域中取出最大值
- Average池化则是计算目标区域的平均值。

### 池化层的特征

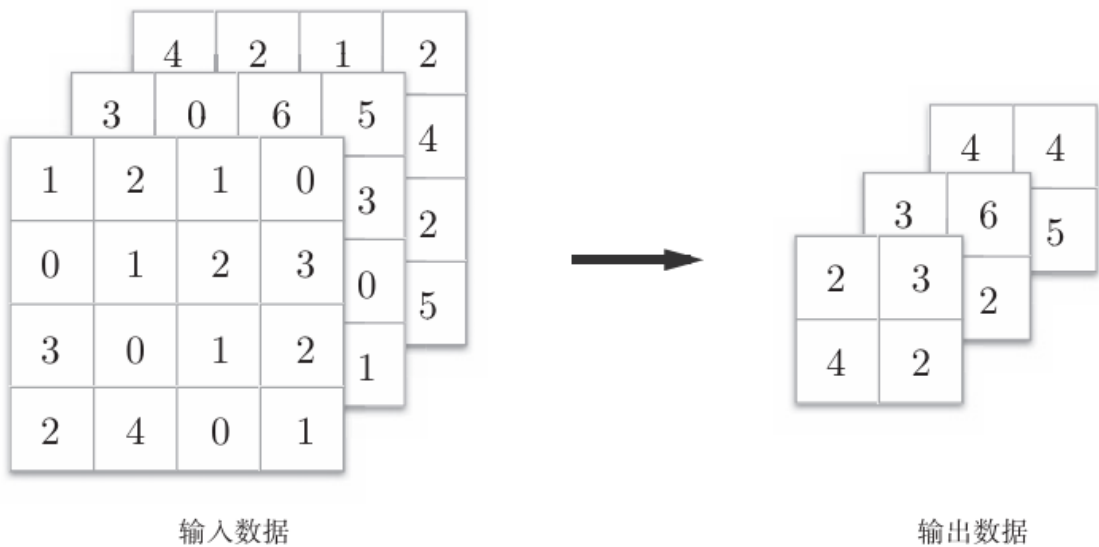
#### (1) 没有要学习的参数



- 池化层和卷积层不同，没有要学习的参数。池化只是从目标区域中取最大值（或者平均值），所以不存在要学习的参数。

### (2) 通道数不发生变化

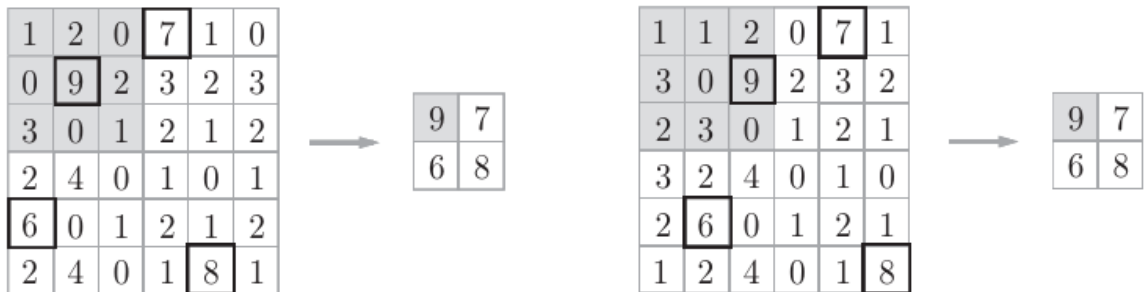
- 经过池化运算，输入数据和输出数据的通道数不会发生变化。如下图所示，计算是按通道独立进行的。



### (3) 对微小的位置变化具有鲁棒性（健壮）

输入数据发生微小偏差时，池化仍会返回相同的结果。因此，池化对 输入数据的微小偏差具有鲁棒性。

- 比如，3×3的池化的情况下，如下图所示，池化会吸收输入数据的偏差（根据数据的不同，结果有可能不一致）。



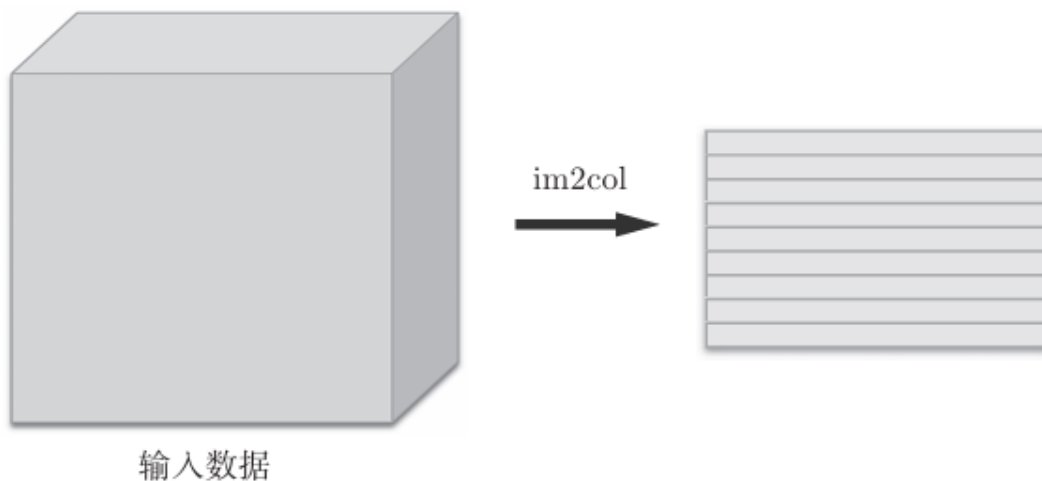
## 6.4 卷积层和池化层的实现

### 6.4.1 基于im2col的展开

im2col是一个函数，将输入数据展开以适合滤波器（权重）。

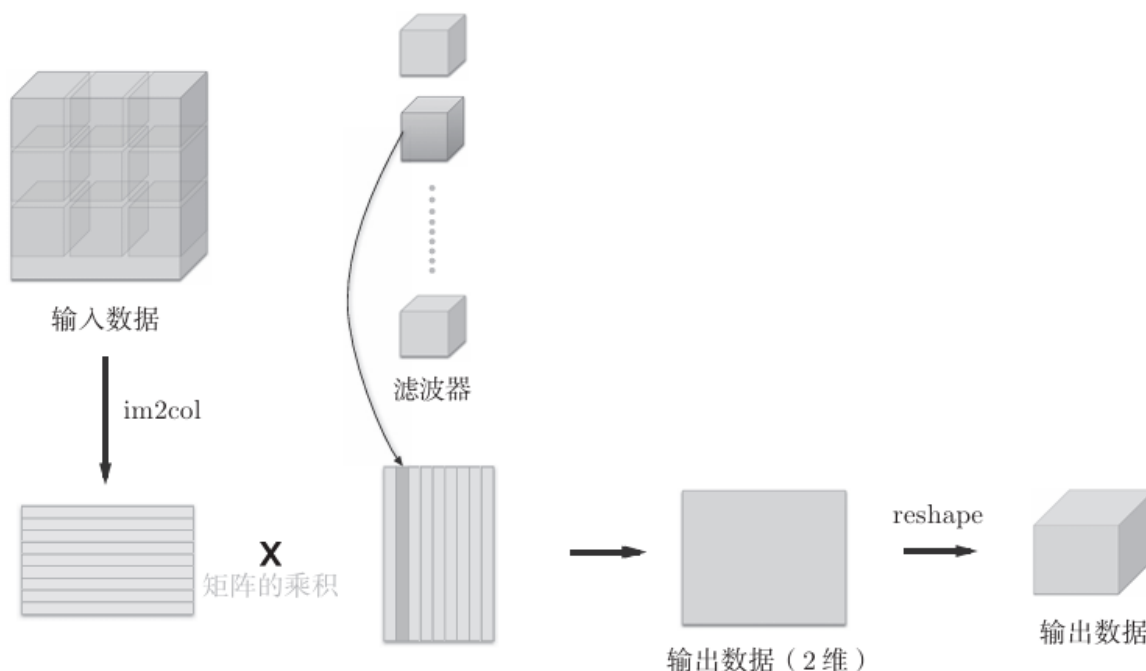
对3维的输入数据应用im2col后，数据转换为2维矩阵（正确地讲，是把包含 批数量的4维数据转换成了2维数据）。

如下图所示



基于`im2col`方式的输出结果是2维矩阵。因为CNN中数据会保存为4维数组，所以要将2维输出数据转换为合适的形状。以上就是卷积层的实现流程。

如下图所示：



卷积运算的滤波器处理的细节：

将滤波器纵向展开为1列，并计算和`im2col`展开的数据的矩阵乘积，最后转换（`reshape`）为输出数据的大小

## 6.4.2 卷积层的实现

`im2col`这一便捷函数具有以下接口

```
im2col (input_data, filter_h, filter_w, stride=1, pad=0)
```

- `input_data`—由（数据量，通道，高，长）的4维数组构成的输入数据
- `filter_h`—滤波器的高
- `filter_w`—滤波器的长
- `stride`—步幅
- `pad`—填充

```

1 import sys, os
2 sys.path.append(os.pardir)
3 from common.util import im2col
4 x1 = np.random.rand(1, 3, 7, 7)
5 col1 = im2col(x1, 5, 5, stride=1, pad=0)
6 print(col1.shape) # (9, 75)
7 x2 = np.random.rand(10, 3, 7, 7) # 10个数据
8 col2 = im2col(x2, 5, 5, stride=1, pad=0)
9 print(col2.shape) # (90, 75)

```

这里举了两个例子。

第一个是批大小为1、通道为3的7×7的数据，第二个的批大小为10，数据形状和第一个相同。

- 分别对其应用im2col函数，在这两种情形下，第2维的元素个数均为75。
- 这是滤波器（通道为3、大小为5×5）的元素个数的总和。
- 批大小为1时，im2col的结果是(9,75)。
- 而第2个例子中批大小为10，所以保存了10倍的数据，即(90,75)。

卷积层实现名为Convolution 的类

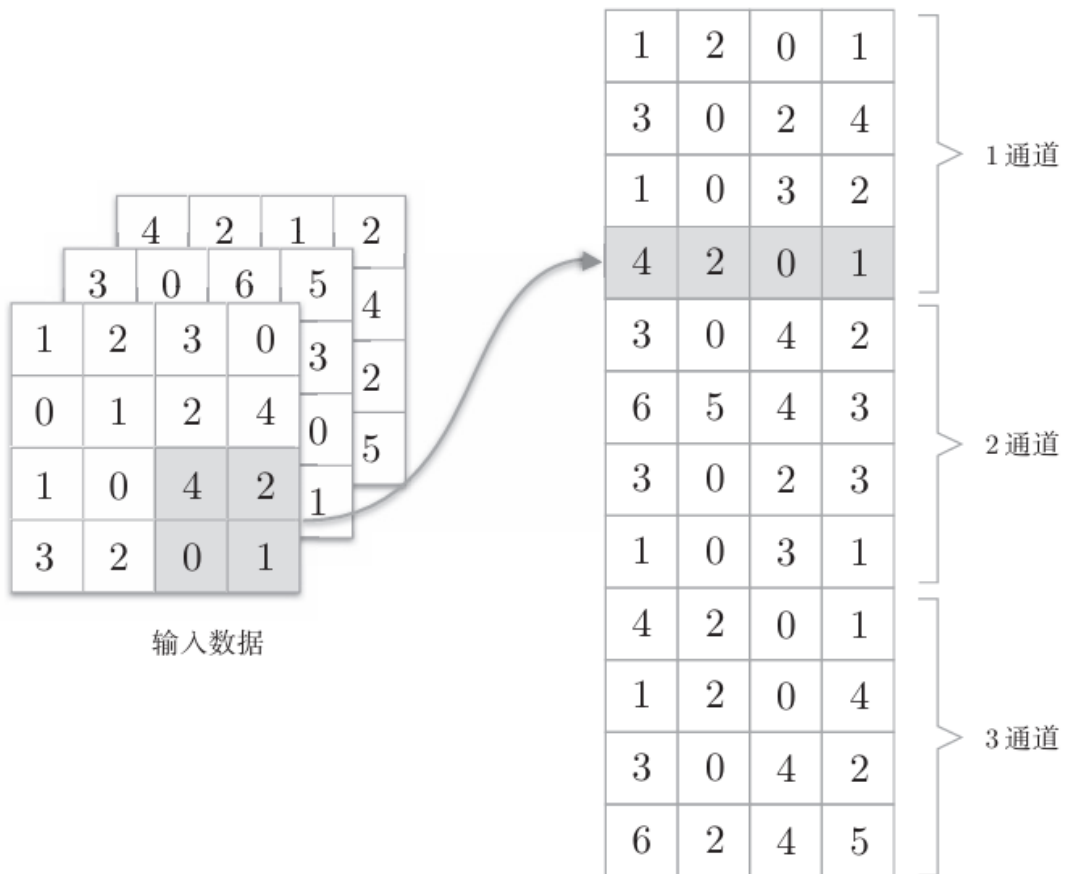
```

1 class Convolution:
2     def __init__(self, w, b, stride=1, pad=0):
3         self.w = w
4         self.b = b
5         self.stride = stride
6         self.pad = pad
7     def forward(self, x):
8         FN, C, FH, FW = self.w.shape
9         N, C, H, W = x.shape
10        out_h = int(1 + (H + 2*self.pad - FH) / self.stride)
11        out_w = int(1 + (W + 2*self.pad - FW) / self.stride)
12        col = im2col(x, FH, FW, self.stride, self.pad)
13        col_w = self.w.reshape(FN, -1).T # 滤波器的展开
14        out = np.dot(col, col_w) + self.b
15        out = out.reshape(N, out_h, out_w, -1).transpose(0, 3, 1, 2)
16        return out
17

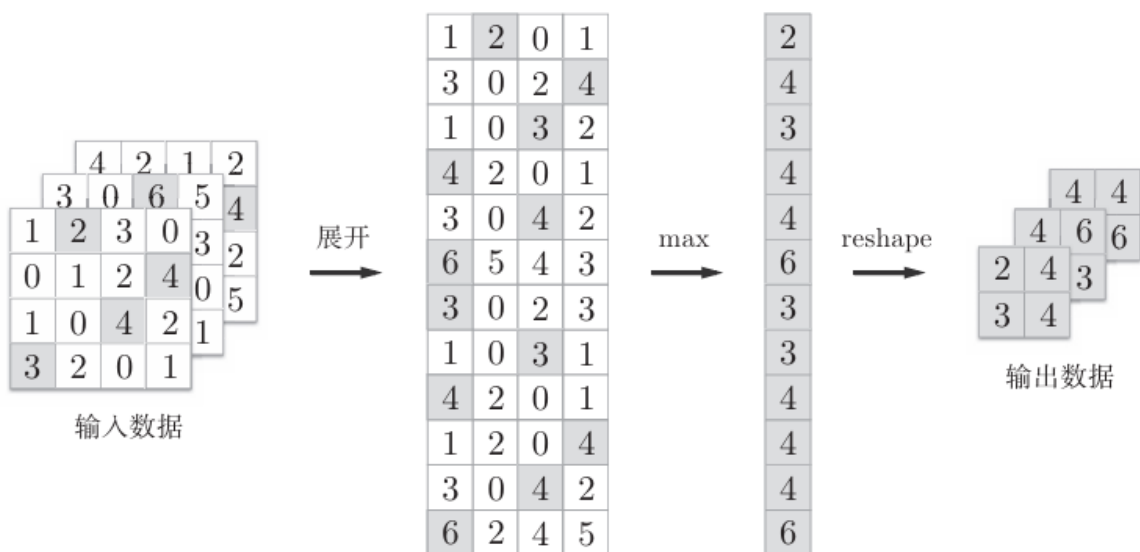
```

## 6.4.3 池化层的实现

池化的应用区域按通道单独展开



池化层的实现流程：池化的应用区域内的最大值元素用灰色表示



池化层的forward实现代码：

```

1  class Pooling:
2      def __init__(self, pool_h, pool_w, stride=1, pad=0):
3          self.pool_h = pool_h
4          self.pool_w = pool_w
5          self.stride = stride
6          self.pad = pad
7      def forward(self, x):
8          N, C, H, W = x.shape
9          out_h = int(1 + (H - self.pool_h) / self.stride)
10         out_w = int(1 + (W - self.pool_w) / self.stride)
11         #

```

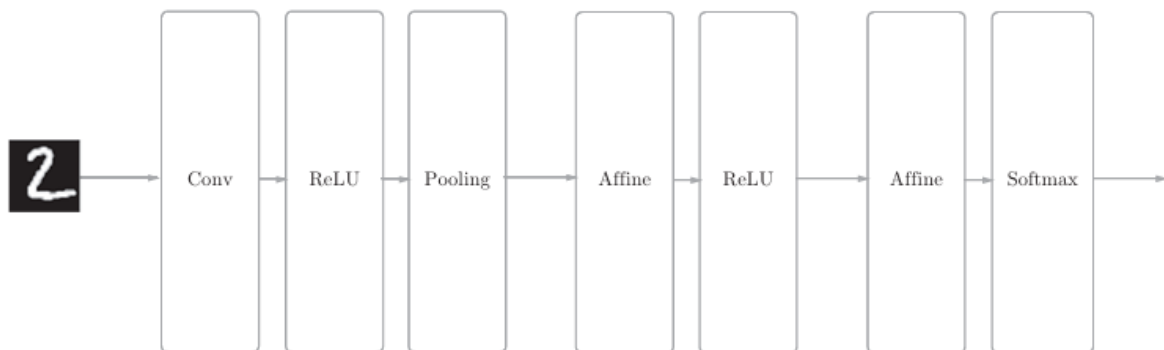
```

12 展开(1)
13     col = im2col(x, self.pool_h, self.pool_w, self.stride, self.pad)
14     col = col.reshape(-1, self.pool_h*self.pool_w)
15     #
16 最大值(2)
17     out = np.max(col, axis=1)
18     #
19 转换(3)
20     out = out.reshape(N, out_h, out_w, C).transpose(0, 3, 1, 2)
21     return out

```

## 6.5 CNN的实现

简单CNN的网络构成



SimpleConvNet的初始化 (\_\_init\_\_)，取下面这些参数

- input\_dim—输入数据的维度：（通道，高，长）
- conv\_param—卷积层的超参数（字典）。字典的关键字如下：
  - filter\_num—滤波器的数量
  - filter\_size—滤波器的大小
  - stride—步幅
  - pad—填充
- hidden\_size—隐藏层（全连接）的神经元数量
- output\_size—输出层（全连接）的神经元数量
- weight\_init\_std—初始化时权重的标准差

SimpleConvNet的初始化的实现稍长，我们分成3部分来说明，首先是初始化的最开始部分

```

1  class SimpleConvNet:
2      def __init__(self, input_dim=(1, 28, 28),
3                    conv_param={'filter_num':30, 'filter_size':5,
4                                'pad':0, 'stride':1},
5                    hidden_size=100, output_size=10, weight_init_std=0.01):
6          filter_num = conv_param['filter_num']
7          filter_size = conv_param['filter_size']
8          filter_pad = conv_param['pad']
9          filter_stride = conv_param['stride']
10         input_size = input_dim[1]
11         conv_output_size = (input_size - filter_size + 2*filter_pad) / \
12                             filter_stride + 1
13         pool_output_size = int(filter_num * (conv_output_size/2) *
14                                 (conv_output_size/2))
15

```

接下来是权重参数的初始化部分。

```
1 self.params = {}
2 self.params['w1'] = weight_init_std * \
3     np.random.randn(filter_num, input_dim[0],
4                       filter_size, filter_size)
5 self.params['b1'] = np.zeros(filter_num)
6 self.params['w2'] = weight_init_std * \
7     np.random.randn(pool_output_size,
8                       hidden_size)
9 self.params['b2'] = np.zeros(hidden_size)
10 self.params['w3'] = weight_init_std * \
11     np.random.randn(hidden_size, output_size)
12 self.params['b3'] = np.zeros(output_size)
```

最后，生成必要的层

```
1 self.layers = OrderedDict()
2 self.layers['Conv1'] = Convolution(self.params['w1'],
3                                     self.params['b1'],
4                                     conv_param['stride'],
5                                     conv_param['pad'])
6 self.layers['Relu1'] = ReLU()
7 self.layers['Pool1'] = Pooling(pool_h=2, pool_w=2, stride=2)
8 self.layers['Affine1'] = Affine(self.params['w2'],
9                                  self.params['b2'])
10 self.layers['Relu2'] = ReLU()
11 self.layers['Affine2'] = Affine(self.params['w3'],
12                                 self.params['b3'])
13 self.last_layer = softmaxwithloss()
```

像这样初始化后，进行推理的predict方法和求损失函数值的loss方法就可以像下面这样实现。

```
1 def predict(self, x):
2     for layer in self.layers.values():
3         x = layer.forward(x)
4     return x
5 def loss(self, x, t):
6     y = self.predict(x)
7     return self.lastLayer.forward(y, t)
```

接下来是基于误差反向传播法求梯度的代码实现

```
1 def gradient(self, x, t):
2     # forward
3     self.loss(x, t)
4     # backward
5     dout = 1
6     dout = self.lastLayer.backward(dout)
7     layers = list(self.layers.values())
8     layers.reverse()
9     for layer in layers:
10         dout = layer.backward(dout)
11     设定
12     #
```

```

13     grads = {}
14     grads['w1'] = self.layers['Conv1'].dw
15     grads['b1'] = self.layers['Conv1'].db
16     grads['w2'] = self.layers['Affine1'].dw
17     grads['b2'] = self.layers['Affine1'].db
18     grads['w3'] = self.layers['Affine2'].dw
19     grads['b3'] = self.layers['Affine2'].db
20     return grads

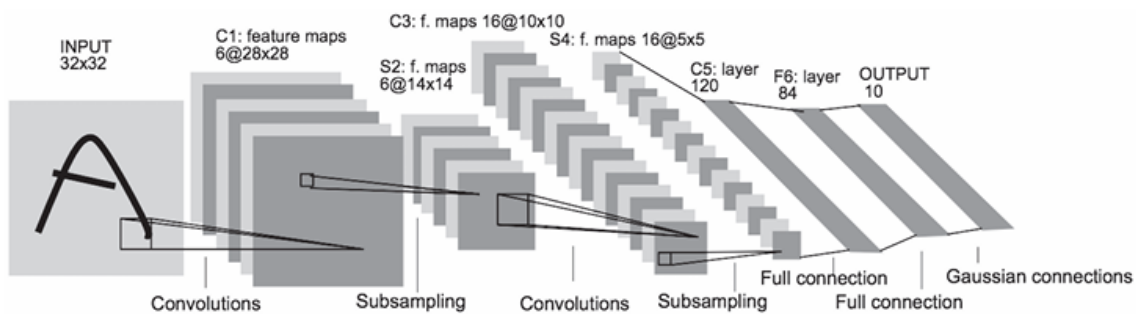
```

## 6.6 具有代表性的CNN

### 6.6.1 LeNet

LeNet在1998年被提出，是进行手写数字识别的网络。

LeNet的网络结构如下图

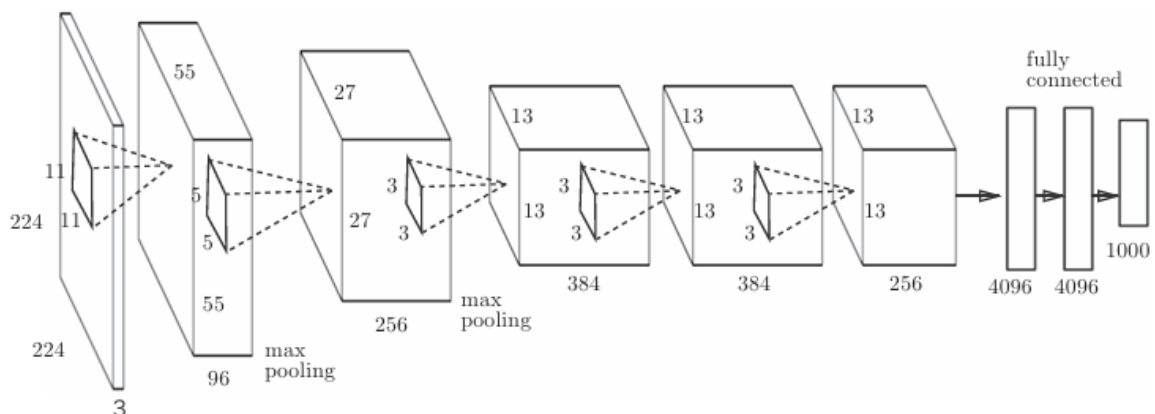


和“现在的CNN”相比，LeNet有几个不同点。

- 第一个不同点在于激活函数。
  - LeNet中使用sigmoid函数，
  - 而现在的CNN中主要使用ReLU函数。
- 此外，原始的LeNet中使用子采样（subsampling）缩小中间数据的大小，而现在的CNN中Max池化是主流。

### 6.6.2 AlexNet

AlexNet网络结构



AlexNet叠有多个卷积层和池化层，最后经由全连接层输出结果。虽然结构上AlexNet和LeNet没有大的不同，

但有以下几点差异。

- 激活函数使用ReLU。
  - 使用进行局部正规化的LRN (Local Response Normalization) 层。
  - 使用Dropout。
-

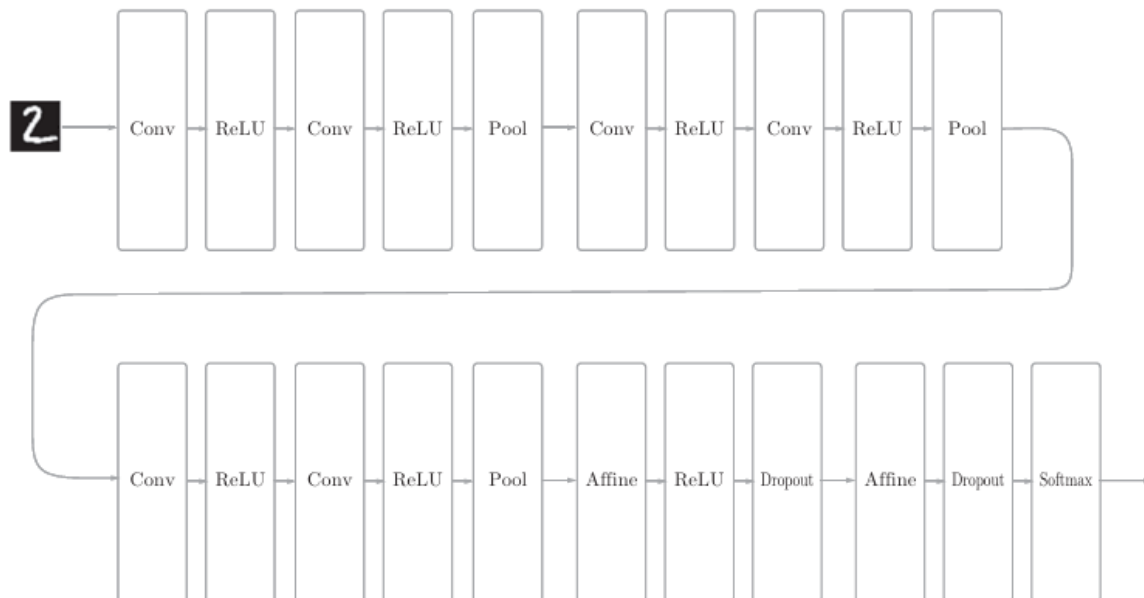


# 第7章 深度学习

深度学习是加深了层的深度神经网络。

## 7.1 加深网络

进行手写数字识别的深度CNN



这个网络有如下特点

- 基于3×3的小型滤波器的卷积层。
- 激活函数是ReLU。
- 全连接层的后面使用Dropout层。
- 基于Adam的最优化。
- 使用He初始值作为权重初始值。

加深层的好处就是可以减少网络的参数数量。

叠加小型滤波器来加深网络的好处是可以减少参数的数量，扩大感受野

## 7.2 深度学习的小历史

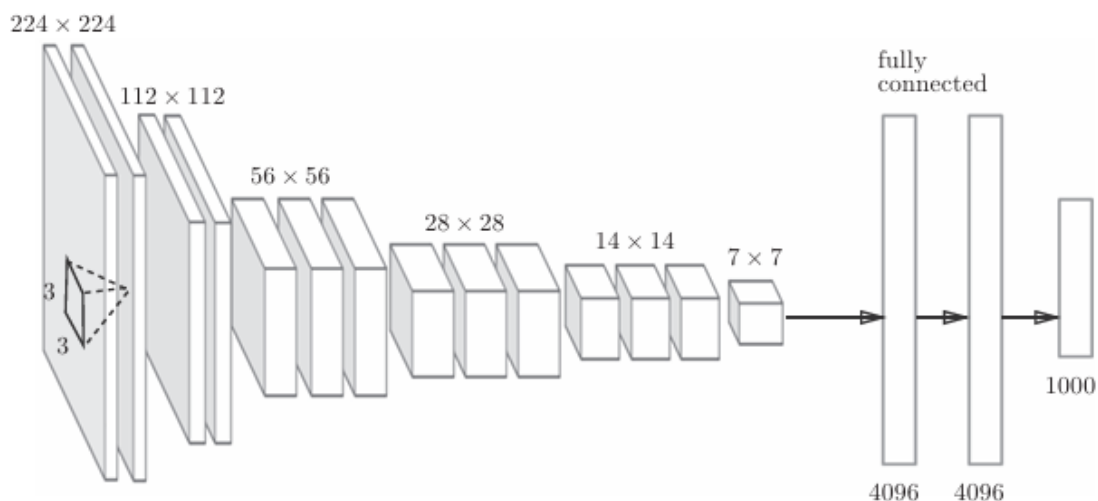
### 7.2.1 ImageNet

ImageNet是拥有超过100万张图像的数据集。

### 7.2.2 VGG

VGG是由卷积层和池化层构成的基础的CNN

不过，如下图所示，它的特点在于将有权重的层（卷积层或者全连接层）叠加至16层（或者19层），具备了深度（根据层的深度，有时也称为“VGG16”或“VGG19”）。



### 7.2.3 GoogLeNet

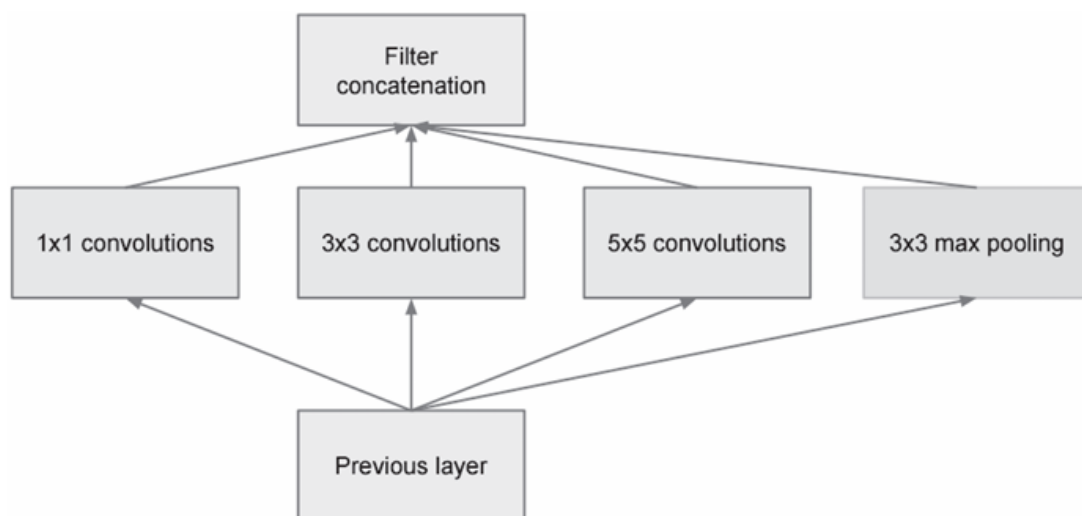
GoogLeNet的特征是，网络不仅在纵向上有深度，在横向上也有深度（广度）

GoogLeNet在横向上有“宽度”，这称为“Inception结构”

Inception结构使用了多个大小不同的滤波器（和池化），最后再合并它们的结果。

GoogLeNet的特征就是将这个Inception结构用作一个构件（构成元素）。

GoogLeNet的Inception结构



### 7.2.4 ResNet

ResNet是微软团队开发的网络。它的特征在于具有比以前的网络更深的结构。

我们已经知道加深层对于提升性能很重要。但是，在深度学习中，过度加深层的话，很多情况下学习将不能顺利进行，导致最终性能不佳。

ResNet中，为了解决这类问题，引入了“快捷结构”（也称为“捷径”或“小路”）。导入这个快捷结构后，就可以随着层的加深而不断提高性能了（当然，层的加深也是有限度的）

实践中经常会灵活应用使用ImageNet这个巨大的数据集学习到的权重数据，这称为**迁移学习**，将学习完的权重（的一部分）复制到其他神经网络，进行再学习（fine tuning）

## 7.3 深度学习的未来

- 图像风格变换
- 图像的生成
- 自动驾驶
- Deep Q-Network (强化学习)